Universidad Distrital Francisco José de Caldas
Ingeniería Electrónica · Análisis y Diseño de Microprocesadores

TALLER 5

Microprocesador de propósito específico: determinante por el algoritmo de Bareiss con salida BCD (algoritmo XS3)

Explicación del diagrama ASM, del camino de datos y del código VHDL

Herramienta: Xilinx ISE 14.7 (XST, ISim) · Tarjeta: Digilent Nexys A7-100T (XC7A100T-1CSG324C)

Nota para el equipo. Este documento explica un diseño de referencia. El enunciado advierte que los diseños parecidos entre equipos no se califican, así que revisen cada estado, entiendan por qué está ahí y adáptenlo antes de entregarlo. Las conclusiones del informe son responsabilidad del equipo.

Contenido

1. Resumen del diseño **3**
- Reglas de descripción . . . 3
- Requisitos del enunciado . . . 3
- Archivos del repositorio . . . 4

2. Repaso del algoritmo de Bareiss **4**
- Ejemplo 3×3 (página 1 de la ROM) . . . 4
- Pivote cero . . . 4

3. Algoritmo XS3 dentro del ASM **4**
- Traza del XS3 para 362 (determinante de la página 3) . . . 5

4. Caja negra **7**
- Tope para la tarjeta . . . 7

5. Cajas grises: unidad de control y camino de datos **8**

6. Pseudocódigo con el hardware disponible **8**

7. Diagrama ASM **10**
- Convenciones . . . 10
- Vista general . . . 10
- Hoja 1: espera y carga de la matriz (s0 a s3) . . . 11
- Hoja 2: pivote, búsqueda de fila e intercambio (s4 a s10) . . . 12
- Hoja 3: productos y numerador (s11 a s16) . . . 13
- Hoja 4: división por restas sucesivas y avance (s17 a s19) . . . 14
- Hoja 5: determinante y valor a convertir (s20 y s21) . . . 15
- Hoja 6: algoritmo XS3 paso a paso, guardado y fin (s22 a s27) . . . 16
- Tabla de estados . . . 17

8. Señales del procesador **19**
- 8.1 Señales de estado (CD → UC) . . . 19
- 8.2 Señales de control por estado (UC → CD) . . . 19
- 8.3 Registros internos y tamaños . . . 20
- Por qué estos anchos . . . 20

9. Camino de datos **21**

10. Tiempo de ejecución **21**

11. Implementación en ISE 14.7 y la Nexys A7 **22**
- 11.1 Proyecto . . . 22
- 11.2 Operación en la tarjeta . . . 22
- 11.3 Matrices de prueba en la ROM . . . 22
- 11.4 Pines (src/nexys_a7.ucf) . . . 23

12. Simulación **23**

13. Limitaciones y verificación pendiente **24**

Anexo A. Código VHDL completo (src/det_bareiss.vhd) **25**

1. Resumen del diseño

El procesador lee de una ROM externa una matriz simétrica de 2×2 a 5×5, la copia a una RAM de trabajo y calcula su determinante con el algoritmo de Bareiss. Usa un multiplicador, un sumador/restador y una división por restas sucesivas, y guarda los pivotes en un banco de registros. Al terminar convierte a BCD, con el algoritmo XS3, el determinante y el pivote que indica `dir_p`.

El ASM tiene 28 estados (s0 a s27) y está escrito en un solo proceso VHDL, con el mismo estilo de los ejemplos de clase (`ASM_contador_unos` y `ASM_Sumax`): un `case st is`, un `when` por estado y las operaciones de cada caja escritas dentro de su estado.

Reglas de descripción

Regla	Cómo se cumple
Solo std_logic (excepto la RAM)	Todas las señales, puertos y constantes son std_logic o std_logic_vector. La única excepción es la RAM, descrita como `array of std_logic_vector` (forma en que XST infiere una memoria); `conv_integer` solo aparece para indexar esa RAM.
Estados del ASM	Los ejemplos de clase usan `type estados_asm is (s0, s1, …)`, que no es std_logic. Para cumplir la regla, los estados son constantes: `constant s0 : STD_LOGIC_VECTOR(4 downto 0) := "00000";`. El `case st is / when s0 =>` se escribe y se lee igual.
Sin variables ni tipos de software	No hay `variable`, `integer`, `signed/unsigned`, `boolean` ni genéricos en el diseño. La ROM es un `with dir select`; el registro BCD es un vector de 40 bits; el banco son 4 registros.
Sintetizable	Sin `wait`, `after` ni lazos `for/while`. La síntesis genérica con GHDL termina sin errores ni latches, con un solo multiplicador.
Solo para simulación	Los testbench sí usan variables, integer, string y wait; no se sintetizan.

Requisitos del enunciado

Requisito del taller	Cómo lo cumple el diseño
ASM en VHDL de alto nivel, con inicio y final	Un proceso con `case st is` (s0 a s27). `final = '1'` solo en s27. El cálculo arranca con `inicio` (BTNC).
Matriz en ROM: 00h = n, luego los a(i,j) como en la Figura 1	ROM externa al ASM (`rom_matrices`), conectada por `dira` (5 bits) y a (6 bits, complemento a 2). Los elementos van por columnas: a11, a21, …, an1, a12, … Tiene 8 páginas de prueba.
Multiplicador, sumador/restador y división por restas sucesivas	Entidades `multiplicador` (con signo, 25 × 25) y `sumador_restador` (50 bits). La división son los registros R y Q y el lazo del estado s17, que usa el restador.
Memoria RAM para las iteraciones	Entidad `ram_trabajo` de 64 × 32 bits; dirección fila & columna.
Pivotes en un banco de registros, visibles con los switches	Registros b1 a b4; `dir_p` (SW2..SW0) elige el pivote y la salida `pivote` lo entrega en BCD.
Determinante en los displays	Salida `det` en BCD; SW3 elige si el display muestra det o pivote.
XS3 evidenciado paso a paso en el ASM	Un estado por paso del documento XS3: s22 (paso 1), s23 (pasos 2 y 5), s24 (paso 3) y s25 (paso 4).

Archivos del repositorio

Archivo	Contenido
src/det_bareiss.vhd	Diseño completo: ROM, RAM, multiplicador, sumador/restador, ASM, sincronizador, display y tope para la tarjeta.
src/nexys_a7.ucf	Pines y restricción de reloj (100 MHz) para ISE.
sim/tb_asm_bareiss.vhd	Testbench del ASM con la ROM externa: 8 matrices y todos sus pivotes.
sim/tb_det_bareiss.vhd	Testbench del tope: lee el display de 7 segmentos (an, seg).
sim/modelo_referencia.py	Modelo en Python: det y pivotes esperados, ciclos y líneas de la ROM.
sim/resultado_ghdl.txt	Salida de las simulaciones.
doc/figuras/*.png, doc/fuentes/*.py	Diagramas y scripts que generan las figuras y este PDF.

2. Repaso del algoritmo de Bareiss

Bareiss es una eliminación gaussiana sin fracciones. En el paso k, cada elemento de la submatriz que está debajo y a la derecha del pivote se actualiza con la fórmula (1) del enunciado:

```
a(i,j) = ( a(k,k)·a(i,j) − a(i,k)·a(k,j) ) / p        para k < i ≤ n,  k < j ≤ n
p = pivote del paso anterior (p = 1 en el primer paso)
```

Después del paso k, el pivote actual a(k,k) pasa a ser p. Tras n − 1 pasos, el determinante queda en a(n,n). La división siempre es exacta: por la identidad de Sylvester, después del paso k cada a(i,j) es el determinante de una submatriz de (k+1)×(k+1) de A, que es un entero. En la simulación el residuo R siempre termina en 0.

Durante el paso k solo cambian los elementos con i > k y j > k. La fila k y la columna k no se modifican, por eso el cálculo se hace sobre la misma RAM, sin una segunda copia de la matriz.

Ejemplo 3×3 (página 1 de la ROM)

```
A = | 2 1 3 |      k = 1, p = 1:  a22 = (2·4 − 1·1)/1 = 7    a23 = (2·2 − 1·3)/1 = 1
    | 1 4 2 |                      a32 = (2·2 − 3·1)/1 = 1    a33 = (2·5 − 3·3)/1 = 1
    | 3 2 5 |      k = 2, p = 2:  a33 = (7·1 − 1·1)/2 = 3   →   det = 3
Pivotes guardados en el banco: pivote(1) = 2, pivote(2) = 7
```

Pivote cero

Si a(k,k) = 0, el ASM busca la primera fila i > k con a(i,k) ≠ 0 (estado s5), intercambia las filas k e i en las columnas k a n (s6 a s9) e invierte el bit `signo`. Al final, det = −a(n,n) si hubo un número impar de intercambios. Si no hay ninguna fila con a(i,k) ≠ 0, det = 0. Las columnas menores que k no se intercambian porque el algoritmo no las vuelve a leer.

3. Algoritmo XS3 dentro del ASM

El documento XS3 describe 5 pasos. En el ASM cada paso es un estado propio, de modo que el diagrama muestra la conversión completa y no una caja que diga «convertir a BCD». La misma secuencia de estados se usa dos veces: primero para el determinante y después para el pivote. El registro `conv_piv` indica cuál de los dos se está convirtiendo, y s26 guarda el resultado en `det` o en `pivote`.

Paso del documento XS3	Estado	Operación en el ASM
1. Inicializar el registro decimal en ceros	s22	bcd ← 0 (40 bits); cont ← 0
2. Colocar el MSB del registro binario en el LSB del registro decimal	s23	bcd(0) ← bin(31)

Paso del documento XS3	Estado	Operación en el ASM
3. Revisar cada dígito: si es ≥ 5 (> 0100), sumarle 3 (0011)	s24	Un dígito por ciclo (d = 0 a 9). Si dig > 0100, el dígito d se reemplaza por dig + 3, calculado en el sumador/restador.
4. Correr a la izquierda y descartar el bit que sale; repetir desde el paso 2	s25	bcd ← bcd(38..0) & '0'; bin ← bin(30..0) & '0'; cont ← cont + 1; vuelve a s23
5. Realizar únicamente el paso 2	s23 con cont = 31	En la pasada 32 por s23 entra el último bit y el ASM va a s26 sin pasar por s24 ni s25.

Por qué se suma 3: si un dígito vale 5 o más, al correrlo a la izquierda quedaría en 10 o más, que no es un dígito BCD válido. Sumar 3 antes del corrimiento equivale a sumar 6 después, que es la corrección BCD, y hace que el exceso pase como acarreo al dígito siguiente.

Tamaños: el registro binario `bin` tiene 32 bits, así que el registro decimal necesita 10 dígitos ($2^{32} - 1 = 4\,294\,967\,295$) y el lazo da 32 pasadas por el paso 2 (cont = 0 a 31). El signo no entra al XS3: en s21 se carga |valor| en `bin` y el signo en neg.

Traza del XS3 para 362 (determinante de la página 3)

$362 = 1\,0110\,1010_2$ tiene 9 bits significativos. En las pasadas cont = 0 a 22 solo entran ceros y el registro decimal sigue en 0, por eso la tabla empieza en cont = 23. La columna «bin(31..23)» muestra los 9 bits de arriba del registro binario; el bit de la izquierda, bin(31), es el que copia el paso 2. El registro decimal se muestra con sus tres dígitos bajos: centenas, decenas y unidades.

cont	Estado (paso)	BCD: C D U	bin(31..23)	Comentario
23	s23 (2)	0000 0000 0001	101101010	
	s24 (3)	0000 0000 0001	101101010	ningún dígito > 4
	s25 (4)	0000 0000 0010	011010100	corrimiento a la izquierda
24	s23 (2)	0000 0000 0010	011010100	
	s24 (3)	0000 0000 0010	011010100	ningún dígito > 4
	s25 (4)	0000 0000 0100	110101000	corrimiento a la izquierda
25	s23 (2)	0000 0000 0101	110101000	
	s24 (3)	0000 0000 1000	110101000	BCD(0) = 0101 > 0100: +3 = 1000
	s25 (4)	0000 0001 0000	101010000	corrimiento a la izquierda
26	s23 (2)	0000 0001 0001	101010000	
	s24 (3)	0000 0001 0001	101010000	ningún dígito > 4
	s25 (4)	0000 0010 0010	010100000	corrimiento a la izquierda
27	s23 (2)	0000 0010 0010	010100000	
	s24 (3)	0000 0010 0010	010100000	ningún dígito > 4
	s25 (4)	0000 0100 0100	101000000	corrimiento a la izquierda
28	s23 (2)	0000 0100 0101	101000000	
	s24 (3)	0000 0100 1000	101000000	BCD(0) = 0101 > 0100: +3 = 1000
	s25 (4)	0000 1001 0000	010000000	corrimiento a la izquierda
29	s23 (2)	0000 1001 0000	010000000	
	s24 (3)	0000 1100 0000	010000000	BCD(1) = 1001 > 0100: +3 = 1100
	s25 (4)	0001 1000 0000	100000000	corrimiento a la izquierda
30	s23 (2)	0001 1000 0001	100000000	
	s24 (3)	0001 1011 0001	100000000	BCD(1) = 1000 > 0100: +3 = 1011

cont	Estado (paso)	BCD: C D U	bin(31..23)	Comentario
	s25 (4)	0011 0110 0010	000000000	corrimiento a la izquierda
31	s23 (2)	0011 0110 0010	000000000	último PASO 2 = PASO 5, va a s26

Resultado: BCD = 0011 0110 0010 = 3 6 2. La simulación muestra « 0000362» en el display para esta matriz.

4. Caja negra

La caja negra sigue los apuntes del profesor: la ROM está fuera del bloque «Algoritmo Bareiss» y se conecta por `dira` y a; las entradas son clk, rst, inicio y dir_p; las salidas son el determinante, el pivote y final; el XS3 forma parte del bloque. Es la entidad `asm_bareiss`.

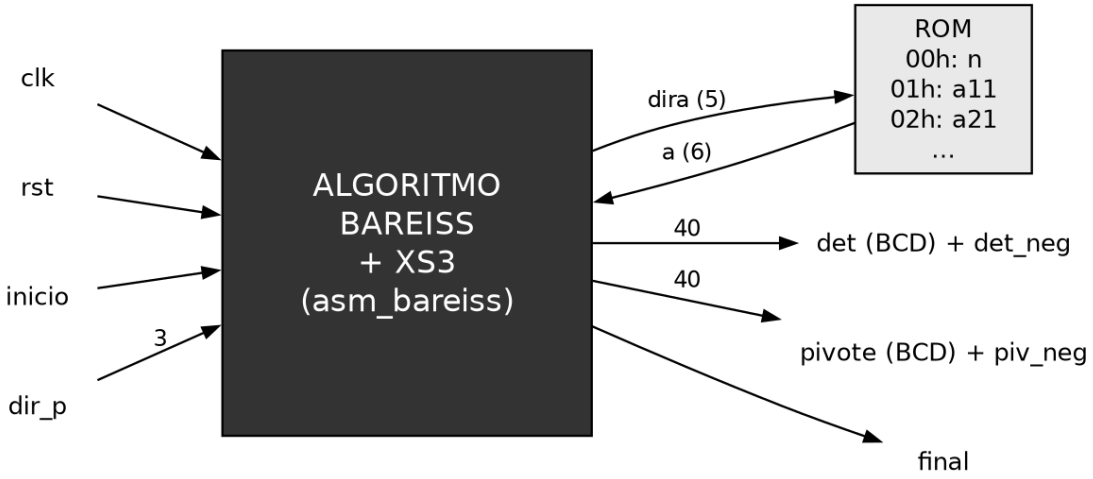


Figura 1. Caja negra del procesador (entidad asm_bareiss) con la ROM externa.

Puerto	Dir.	Bits	Función
clk	in	1	Reloj
rst	in	1	Reinicio asíncrono, activo en alto
inicio	in	1	Comienza el cálculo (se lee en s0 y s27)
dir_p	in	3	Dirección del banco de pivotes: 1 a 4 = pivote k; 0 y 5 a 7 dan 0
a	in	6	Dato de la ROM, complemento a 2
dira	out	5	Dirección de la ROM
det, det_neg	out	40, 1	|det| en BCD (10 dígitos) y su signo
pivote, piv_neg	out	40, 1	|pivote(dir_p)| en BCD y su signo
final	out	1	'1' cuando det y pivote están listos (estado s27)

Tope para la tarjeta

La entidad `det_bareiss` conecta la caja negra con la Nexys A7: sincroniza los botones y switches, conecta la ROM (dirección = SW15..SW13 & dira) y muestra det o pivote en los 8 displays según SW3.

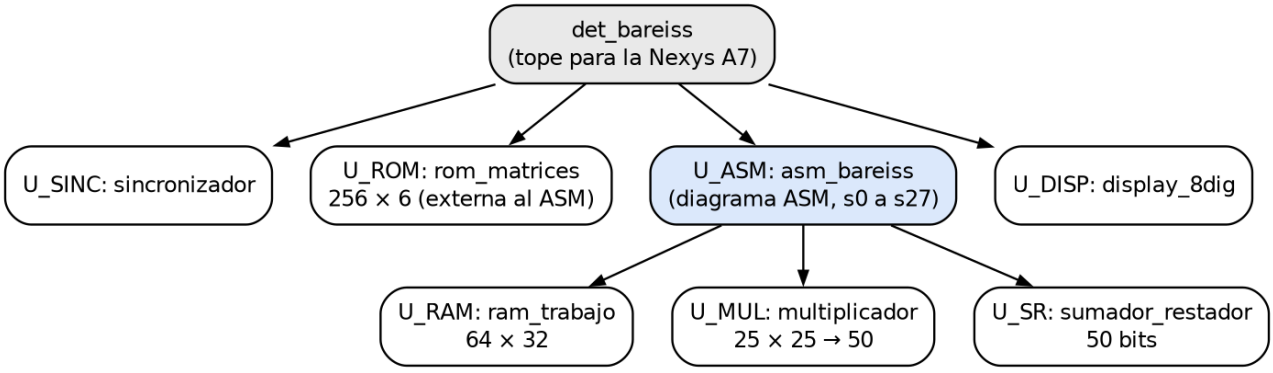


Figura 2. Entidades del archivo src/det_bareiss.vhd.

Puerto del tope	Pin Nexys A7	Función
clk	E3	Reloj de 100 MHz
rst / inicio	BTNU (M18) / BTNC (N17)	Reinicio / inicio del cálculo
sw_mat(2..0)	SW15..SW13	Matriz de prueba (página de la ROM)
dir_p(2..0)	SW2..SW0	Dirección del pivote
ver	SW3	0: el display muestra det; 1: muestra pivote
final / desborde	LD0 / LD1	Fin del cálculo / el valor mostrado tiene más de 7 dígitos
an, seg, dp	AN7..AN0, CA..CG, DP	Display: AN7 = signo, AN6..AN0 = 7 dígitos

5. Cajas grises: unidad de control y camino de datos

La unidad de control (UC) es el registro de estado `st` y la lógica que elige el siguiente estado. El camino de datos (CD) contiene la RAM, los registros y las unidades funcionales. En el VHDL de alto nivel las dos partes están en el mismo proceso, como en los ejemplos de clase; las señales de control son el estado actual y las de estado son las condiciones de las cajas de decisión (sección 8).

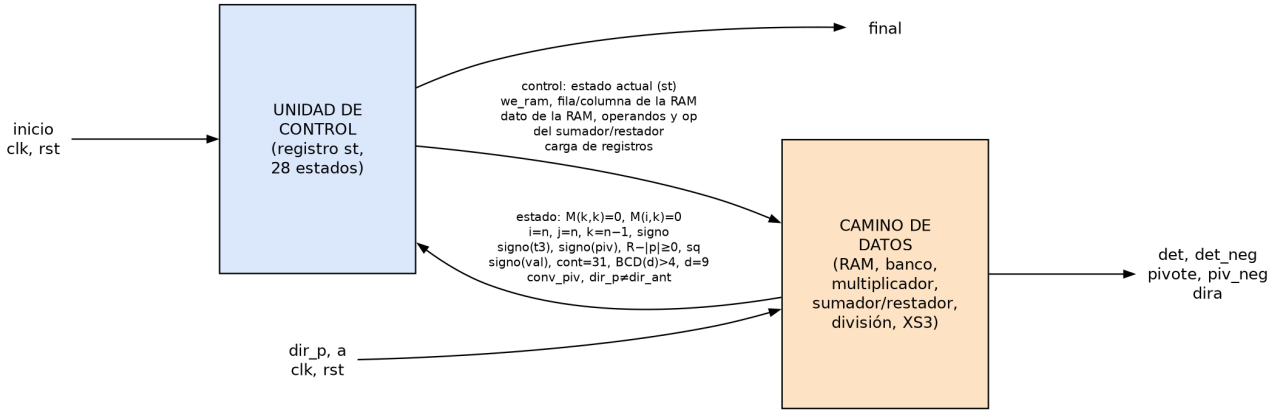


Figura 3. Cajas grises: señales de control (UC → CD) y señales de estado (CD → UC).

6. Pseudocódigo con el hardware disponible

Cada línea indica el estado del ASM que la ejecuta. Las operaciones del mismo estado ocurren en el mismo ciclo de reloj.

```
s0 :  final = 0;  esperar inicio = 1
```

```
s1 :  n = ROM(00h)
s2 :  para j = 1..n, i = 1..n:  M(i,j) = ROM(dira);  dira = dira + 1     (orden por columnas)
s3 :  k = 1;  p = 1;  |p| = 1;  signo = 0;  banco = 0;  conv_piv = 0
mientras k <= n-1:
  s4 :  piv = M(k,k)
        si piv = 0:
  s5 :     buscar i = k+1..n con M(i,k) /= 0
           si no existe:  det_bin = 0;  ir a CONVERTIR
  s6-s9:   intercambiar filas k e i (columnas k..n);  signo = not signo;  volver a s4
  s10:  banco(k) = piv
        para i = k+1..n, j = k+1..n:
  s11-s14:  t1 = piv · M(i,j);   t2 = M(i,k) · M(k,j)        (un multiplicador, dos usos)
  s15:      t3 = t1 - t2                                     (sumador/restador)
  s16-s17:  Q = |t3| / |p|  por restas sucesivas            (sumador/restador)
  s18:      M(i,j) = +Q  o  -Q   (signo = signo(t3) xor signo(p))
  s19:  p = piv;  |p| = |piv|;  k = k + 1
s20:  det_bin = M(n,n);  si signo = 1 entonces det_bin = -det_bin
CONVERTIR (dos veces: conv_piv = 0 para det, conv_piv = 1 para el pivote):
s21:  val = det_bin o banco(dir_p);  bin = |val|;  neg = signo(val)
s22:  XS3 paso 1:  BCD = 0;  cont = 0
repetir:
  s23:  XS3 paso 2:  BCD(0) = bin(31);   si cont = 31 salir   (paso 5)
  s24:  XS3 paso 3:  para d = 0..9:  si BCD(d) > 4 entonces BCD(d) = BCD(d) + 3
  s25:  XS3 paso 4:  BCD = BCD << 1;  bin = bin << 1;  cont = cont + 1
s26:  si conv_piv = 0:  det = BCD;  conv_piv = 1;  ir a s21
      si no:            pivote = BCD
s27:  final = 1;  si inicio = 1 ir a s1;  si cambia dir_p ir a s21
```

Los «para» y «mientras» del pseudocódigo no son lazos del VHDL: cada uno es un lazo del ASM que vuelve a un estado mientras un contador no llegue a su límite.

7. Diagrama ASM

Convenciones

Elemento	Significado
Caja de estado (rectángulo azul)	Operaciones que se hacen en todo el estado (salidas tipo Moore).
Caja de decisión (rombo)	Pregunta sobre una señal de estado. La salida 1 es la condición verdadera.
Caja de salida condicional (rectángulo redondeado naranja)	Operación que solo ocurre si se toma ese camino (tipo Mealy), como en el ejemplo «ASM Mealy» de clase.
Bloque ASM (recuadro punteado)	Caja de estado más sus decisiones y salidas condicionales: dura un ciclo de reloj.
Conector (círculo verde)	Une hojas del diagrama: A → s1, B → s4, C → s11, D → s21, E → s20, F → s17, G → s22.
x ← expresión	Asignación a un registro; se hace efectiva al final del ciclo (flanco de subida).
x = expresión	Valor combinacional (no se guarda), por ejemplo final = 1 o la selección val.
M(i,j)	Palabra de la RAM en la dirección fila i, columna j.

Todas las decisiones de un bloque usan los valores que tienen los registros al comenzar el ciclo. Por ejemplo, en s19 la pregunta k = n − 1 usa el k anterior al incremento. Cada combinación de estado y decisiones lleva a un solo estado siguiente, y todo camino de decisiones termina en un estado.

Vista general

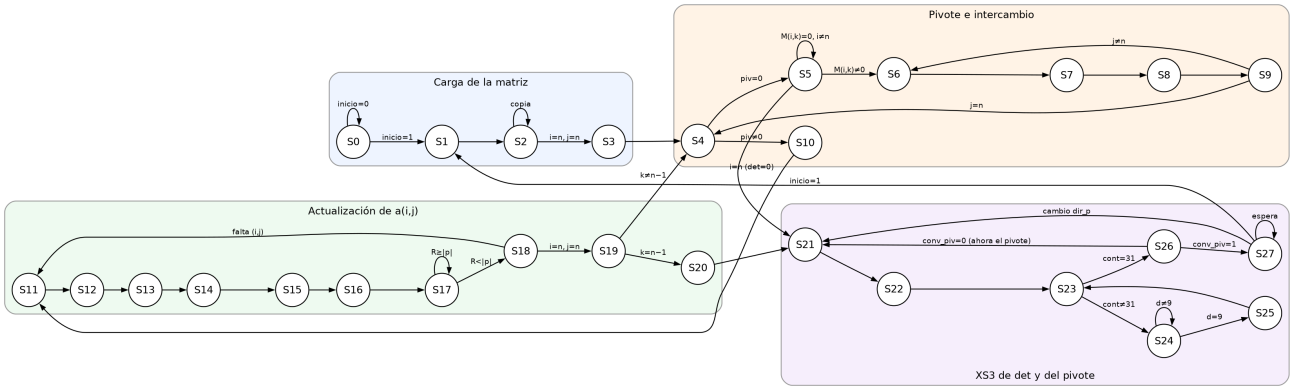


Figura 4. Diagrama de estados del ASM (28 estados), agrupado por función.

Hoja 1: espera y carga de la matriz (s0 a s3)

s2 recorre la ROM externa con el contador `dira` y los contadores de fila y columna. La dirección de la RAM es fila & columna, así que no hace falta multiplicar para ubicar a(i,j).

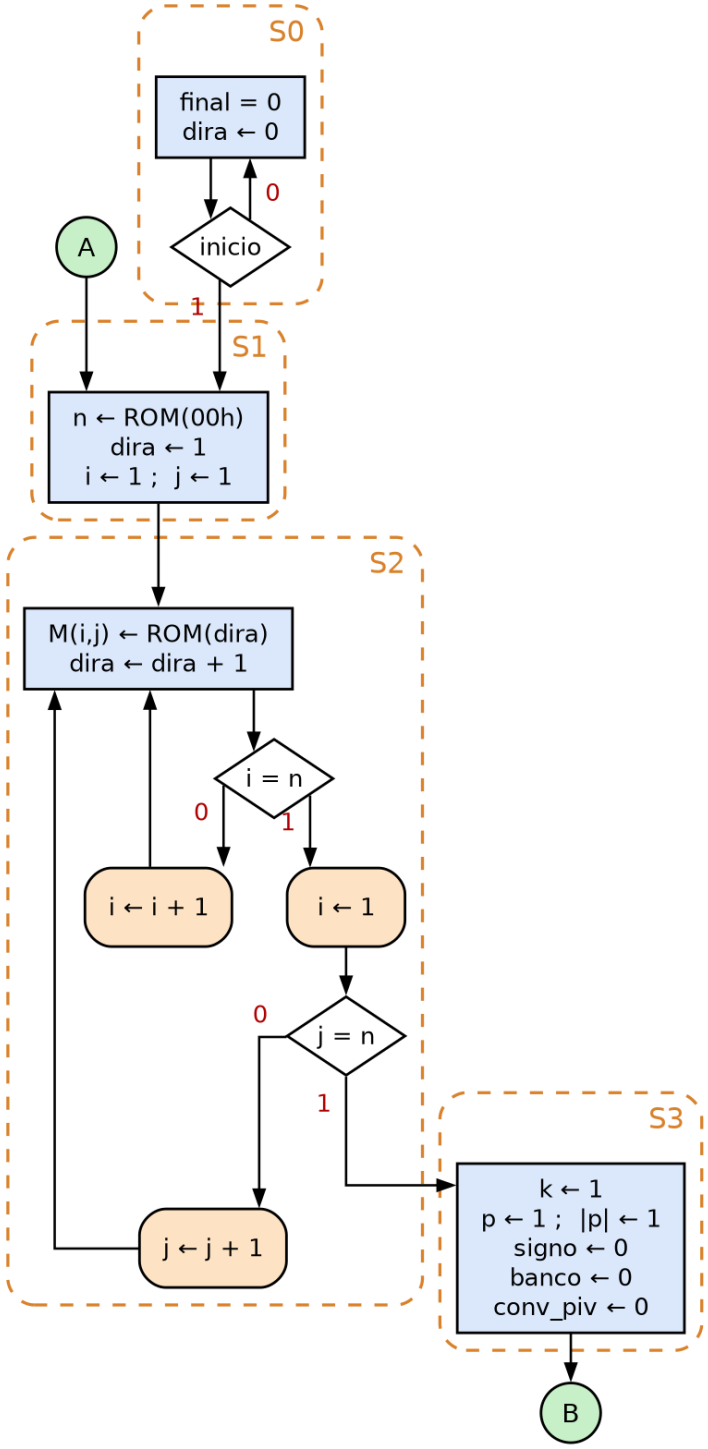


Figura 5. Hoja 1: espera y carga de la matriz (s0 a s3).

Hoja 2: pivote, búsqueda de fila e intercambio (s4 a s10)

La RAM tiene un solo puerto: cada lectura o escritura ocupa un estado, por eso el intercambio usa cuatro estados por columna. Después del intercambio se vuelve a s4 para leer el pivote nuevo, que ya no es cero.

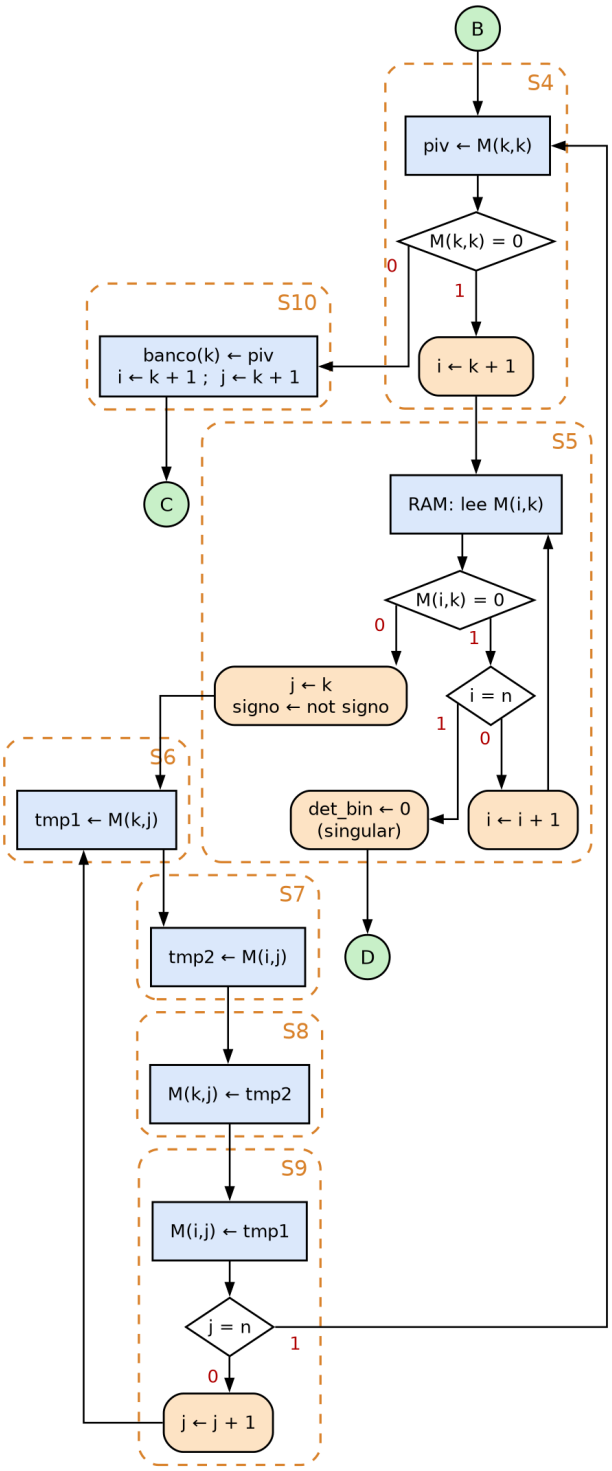


Figura 6. Hoja 2: pivote, búsqueda de fila e intercambio (s4 a s10).

Hoja 3: productos y numerador (s11 a s16)

Con un solo multiplicador, los dos productos de la fórmula (1) se hacen en estados distintos. Los operandos se cargan en mul_a y mul_b un estado antes, para que el camino crítico sea registro → multiplicador → registro.

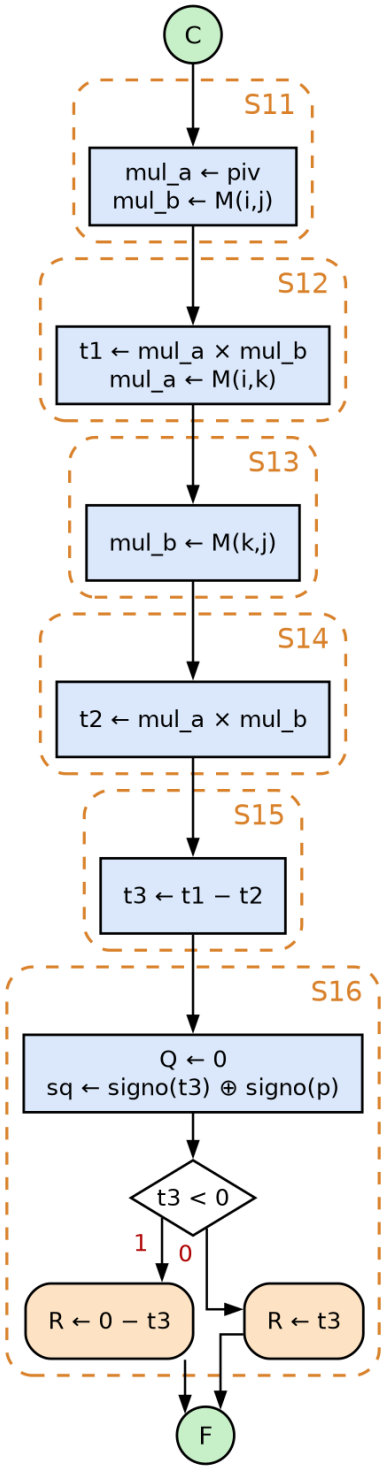


Figura 7. Hoja 3: productos y numerador (s11 a s16).

Hoja 4: división por restas sucesivas y avance (s17 a s19)

s17 es el lazo de la división: en cada ciclo el restador calcula R − |p|; si el resultado no es negativo se guarda en R y el cociente Q aumenta en 1. La división se hace con magnitudes y el signo se aplica al escribir en s18.

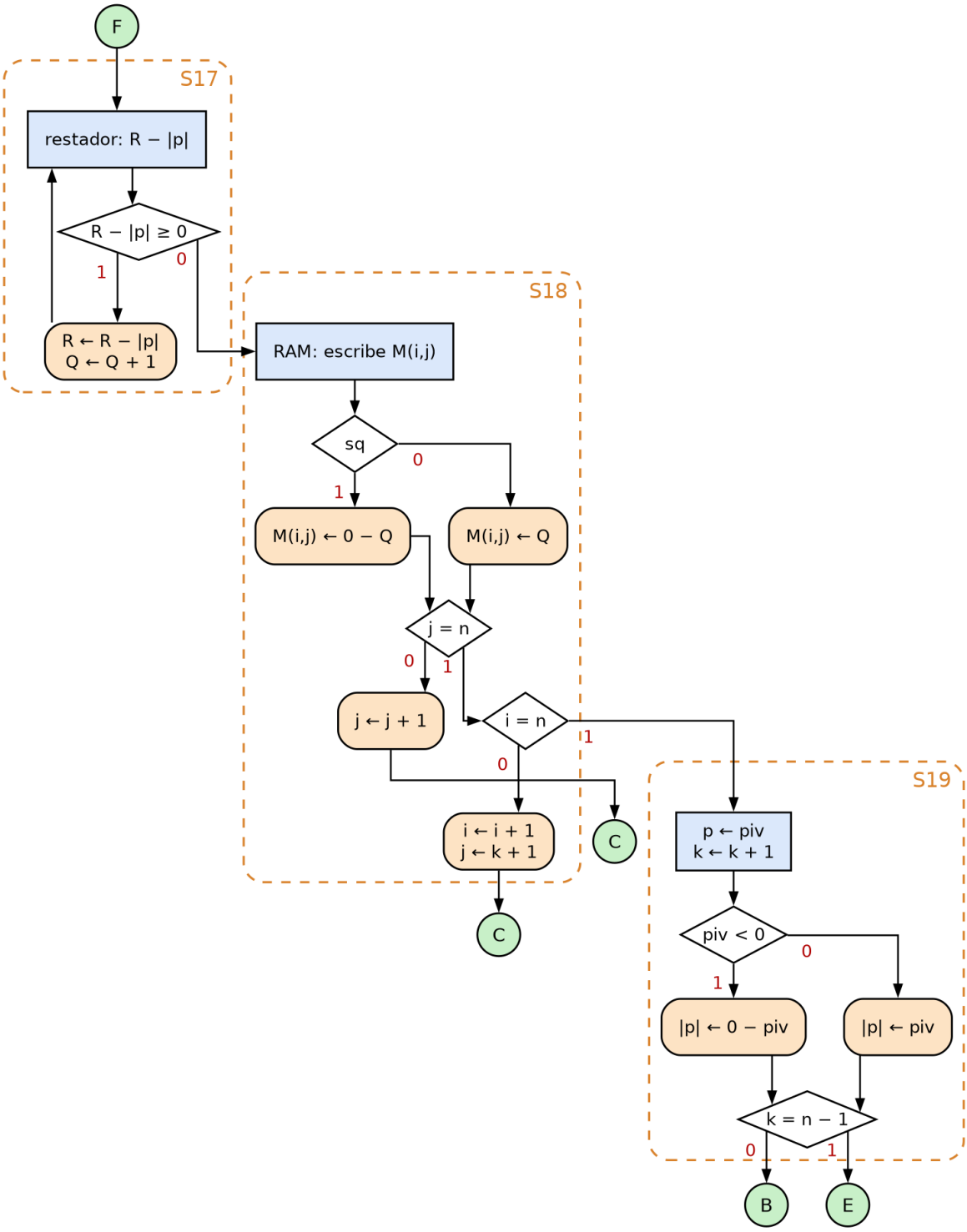


Figura 8. Hoja 4: división por restas sucesivas y avance (s17 a s19).

Hoja 5: determinante y valor a convertir (s20 y s21)

A s21 se llega desde s20 (cálculo normal), desde s5 (matriz singular, det_bin = 0), desde s26 (después de convertir det, para convertir el pivote) y desde s27 cuando cambia dir_p.

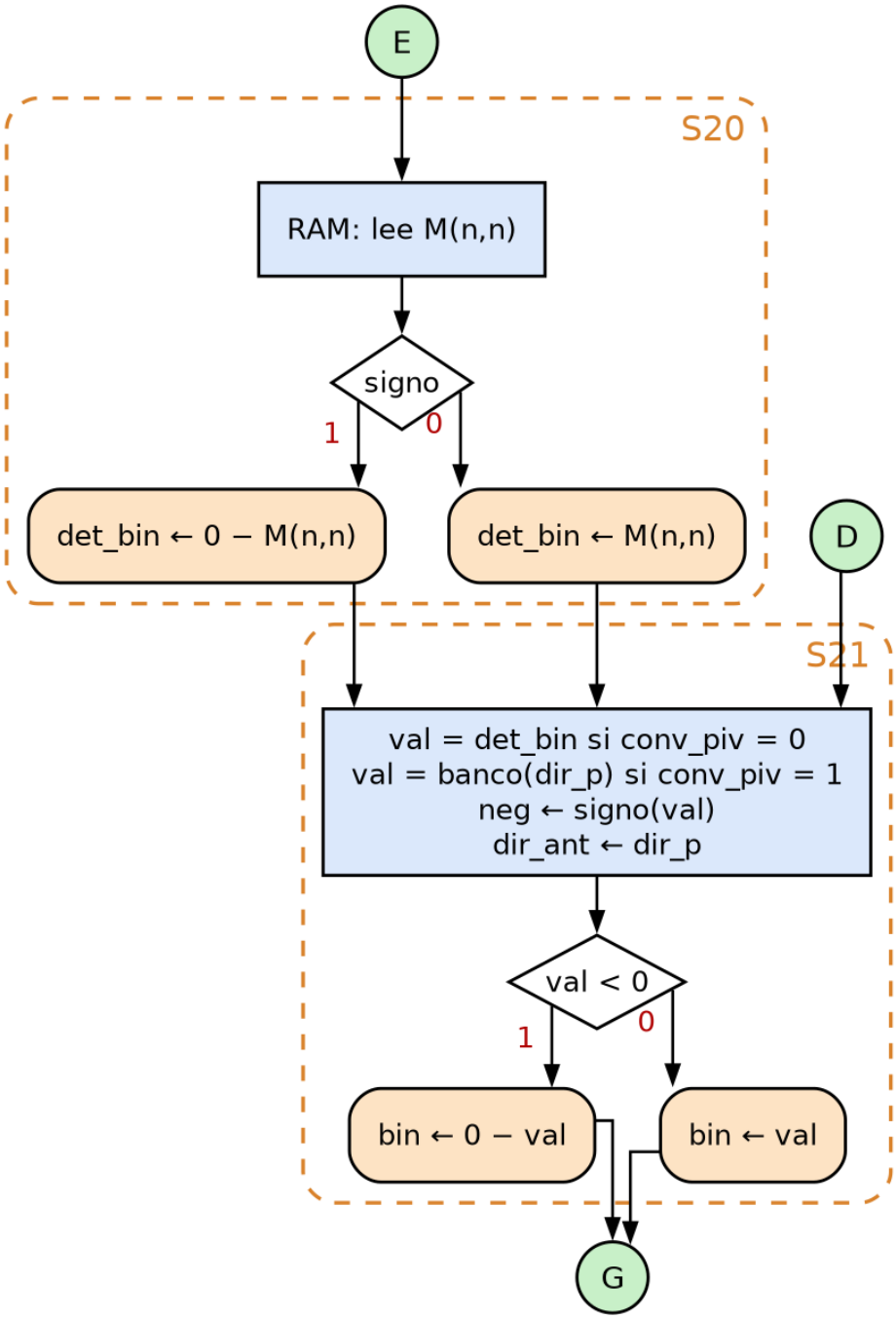


Figura 9. Hoja 5: determinante y valor a convertir (s20 y s21).

Hoja 6: algoritmo XS3 paso a paso, guardado y fin (s22 a s27)

En la primera pasada s26 guarda el BCD en det y vuelve a s21 para convertir el pivote; en la segunda lo guarda en pivote y pasa a s27. s27 espera con final = 1. Si cambia dir_p, el ASM vuelve a s21 y convierte el nuevo pivote (unos 375 ciclos, invisibles para el ojo).

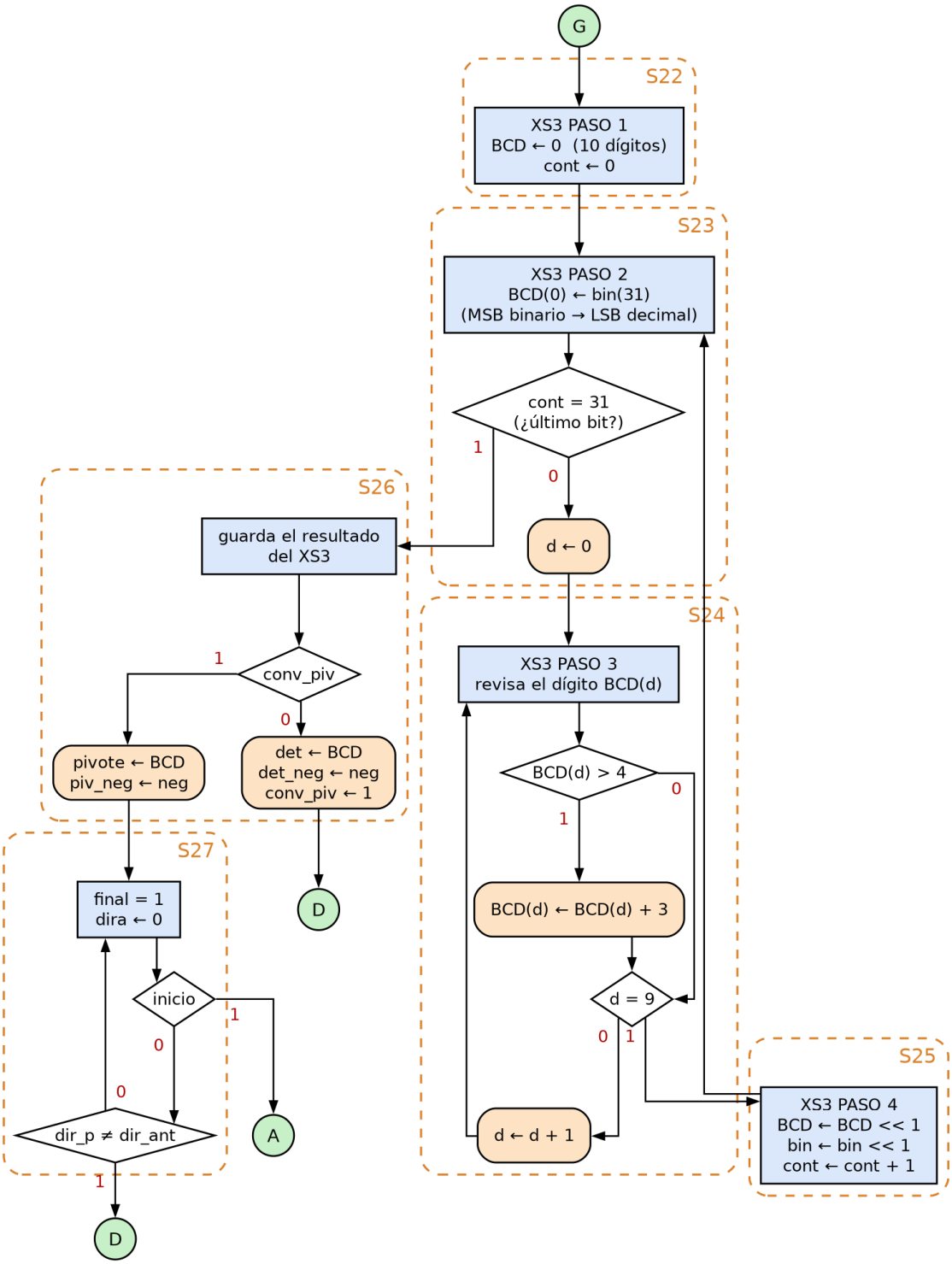


Figura 10. Hoja 6: algoritmo XS3 paso a paso, guardado y fin (s22 a s27).

Tabla de estados

Resumen de cada bloque ASM: lo que hace la caja de estado y las salidas condicionales, y a qué estado va según las decisiones. Es la misma información de las figuras 5 a 10, en el orden del código VHDL.

Estado	Operaciones	Siguiente estado
s0	final = 0; dira ← 0	inicio = 1: s1. Si no: s0.
s1	n ← a(2..0) = ROM(00h); dira ← 1; i ← 1; j ← 1	s2
s2	M(i,j) ← a con extensión de signo a 32 bits; dira ← dira + 1. i es el índice que cambia más rápido, como en la Figura 1 del taller.	i ≠ n: i ← i + 1, s2. i = n y j ≠ n: i ← 1, j ← j + 1, s2. i = n y j = n: s3.
s3	k ← 1; p ← 1 (pivote inicial a(0,0) = 1); |p| ← 1; signo ← 0; banco ← 0; conv_piv ← 0	s4
s4	piv ← M(k,k)	M(k,k) = 0: i ← k + 1, s5. Si no: s10.
s5	Lee M(i,k) buscando una fila con elemento distinto de cero en la columna k.	M(i,k) ≠ 0: j ← k, signo ← not signo, s6. M(i,k) = 0 e i = n: det_bin ← 0, s21 (singular). Si no: i ← i + 1, s5.
s6	tmp1 ← M(k,j)	s7
s7	tmp2 ← M(i,j)	s8
s8	M(k,j) ← tmp2	s9
s9	M(i,j) ← tmp1	j ≠ n: j ← j + 1, s6. j = n: s4 (vuelve a leer el pivote).
s10	banco(k) ← piv; i ← k + 1; j ← k + 1	s11
s11	mul_a ← piv; mul_b ← M(i,j)	s12
s12	t1 ← mul_a × mul_b (= a(k,k)·a(i,j)); mul_a ← M(i,k)	s13
s13	mul_b ← M(k,j)	s14
s14	t2 ← mul_a × mul_b (= a(i,k)·a(k,j))	s15
s15	t3 ← t1 − t2	s16
s16	Q ← 0; sq ← signo(t3) xor signo(p); R ← |t3| (0 − t3 si t3 < 0)	s17
s17	El restador calcula R − |p| (resta de prueba).	R − |p| ≥ 0: R ← R − |p|, Q ← Q + 1, s17. Si no: s18 (R queda en 0 porque la división es exacta).
s18	M(i,j) ← Q, o 0 − Q si sq = 1	j ≠ n: j ← j + 1, s11. j = n e i ≠ n: i ← i + 1, j ← k + 1, s11. i = n y j = n: s19.
s19	p ← piv; |p| ← |piv|; k ← k + 1	k = n − 1: s20. Si no: s4.
s20	det_bin ← M(n,n), o 0 − M(n,n) si signo = 1	s21
s21	val = det_bin si conv_piv = 0, banco(dir_p) si conv_piv = 1; neg ← signo(val); bin ← |val|; dir_ant ← dir_p	s22
s22	XS3 paso 1: bcd ← 0; cont ← 0	s23
s23	XS3 paso 2: bcd(0) ← bin(31)	cont = 31: s26 (paso 5 cumplido). Si no: d ← 0, s24.
s24	XS3 paso 3 sobre el dígito d: si dig > 4, dígito d ← dig + 3 (en el sumador/restador)	d ≠ 9: d ← d + 1, s24. d = 9: s25.
s25	XS3 paso 4: bcd ← bcd << 1 (entra 0); bin ← bin << 1; cont ← cont + 1	s23
s26	Guarda el resultado del XS3.	conv_piv = 0: det ← bcd, det_neg ← neg, conv_piv ← 1, s21. conv_piv = 1: pivote ← bcd, piv_neg ← neg, s27.

Estado	Operaciones	Siguiente estado
s27	final = 1; dira ← 0	inicio = 1: s1. dir_p ≠ dir_ant: s21. Si no: s27.

8. Señales del procesador

8.1 Señales de estado (CD → UC)

Señal de estado	Cómo se obtiene	Se pregunta en
inicio	Botón BTNC sincronizado con dos flip-flops	s0, s27
M(k,k) = 0	Comparador de dout_ram con 0	s4
M(i,k) = 0	Comparador de dout_ram con 0	s5
i = n	Comparador de 3 bits	s2, s5, s18
j = n	Comparador de 3 bits	s2, s9, s18
k = n − 1	Comparador k + 1 = n	s19
signo(t3)	Bit t3(49)	s16
R − |p| ≥ 0	Bit 49 de la salida del sumador/restador en 0	s17
sq	Registro de signo del cociente	s18
signo(piv)	Bit piv(31)	s19
signo	Registro con la paridad de intercambios de filas	s20
signo(val)	Bit val(31)	s21
cont = 31	Comparador de 5 bits	s23
BCD(d) > 4	Comparador de 4 bits sobre el dígito elegido por d	s24
d = 9	Comparador de 4 bits	s24
conv_piv	Registro: 0 = se convierte det, 1 = se convierte el pivote	s26 (y el mux val en s21)
dir_p ≠ dir_ant	Comparador de 3 bits (dir_p sincronizado)	s27

8.2 Señales de control por estado (UC → CD)

Además de cargar los registros que aparecen en cada caja, el estado controla la RAM, el multiplicador y el sumador/restador. Los estados que no aparecen no usan estas unidades.

Estado	Dirección RAM (fila, col)	Escribe en RAM	Sumador/restador	Multiplicador
s2	(i, j)	ROM con extensión de signo	—	—
s4	(k, k)	—	—	—
s5	(i, k)	—	—	—
s6	(k, j)	—	—	—
s7	(i, j)	—	—	—
s8	(k, j)	tmp2	—	—
s9	(i, j)	tmp1	—	—
s11	(i, j)	—	—	carga mul_a, mul_b
s12	(i, k)	—	—	t1 ← producto; carga mul_a
s13	(k, j)	—	—	carga mul_b
s14	—	—	—	t2 ← producto
s15	—	—	t1 − t2	—
s16	—	—	0 − t3	—
s17	—	—	R − |p|	—

Estado	Dirección RAM (fila, col)	Escribe en RAM	Sumador/restador	Multiplicador
s18	(i, j)	Q, o la salida del restador si sq = 1	0 − Q	—
s19	—	—	0 − piv	—
s20	(n, n)	—	0 − M(n,n)	—
s21	—	—	0 − val	—
s24	—	—	dig + 3 (suma)	—

En el VHDL estas señales son asignaciones concurrentes que dependen de `st`: `fila_ram`, `col_ram`, `we_ram`, `din_ram`, `sr_a`, `sr_b` y `sr_op`. Si en el informe implementan la UC con ROM de entrada, registro y ROM de salida, como en el ejemplo del contador de unos, esta tabla es la base de la ROM de salida (palabra de control).

8.3 Registros internos y tamaños

Registro	Bits	Uso
st	5 (28 estados)	Estado del ASM
dira_r	5	Dirección de la ROM (sale por dira)
n, i, j, k	3 c/u	Tamaño de la matriz y contadores de fila, columna y paso
piv, p, pabs	32 c/u	Pivote actual, pivote anterior (divisor) y su magnitud
tmp1, tmp2	32 c/u	Intercambio de filas
mul_a, mul_b	25 c/u	Registros de entrada del multiplicador
t1, t2, t3	50 c/u	Productos y numerador de la fórmula (1)
R, Q, sq	50, 32, 1	Residuo, cociente y signo del cociente
signo	1	Paridad de intercambios de filas
det_bin	32	Determinante en binario
b1, b2, b3, b4	32 c/u	Banco de pivotes
conv_piv, dir_ant	1, 3	Qué se convierte y qué dir_p ya se convirtió
bin, bcd	32, 40	Registros binario y decimal del XS3
cont, d, neg	5, 4, 1	Bits procesados, dígito revisado y signo del valor convertido
det_bcd, det_neg_r	40, 1	Salida det
piv_bcd, piv_neg_r	40, 1	Salida pivote

Por qué estos anchos

Por la identidad de Sylvester, cada valor guardado en la RAM es el determinante de una submatriz m×m de A. La desigualdad de Hadamard lo acota: $|\det| \le (32\cdot\sqrt{m})^m$ cuando $|a(i,j)| \le 32$.

- Operandos del multiplicador: en el paso k son submatrices de k×k con $k \le 4$, así que $|x| < 2^{24} = 16\,777\,216$. Caben en 25 bits con signo. (La cota no se alcanza porque +32 no existe en 6 bits.)
- Determinante 5×5: $|\det| \le 1{,}88 \times 10^9 < 2^{31}$. Por eso la RAM, det_bin, Q y bin son de 32 bits.
- Numerador: t3 = p · a(k), con $|p| \le 170\,268$ (submatriz 3×3), así que $|t3| \le 3{,}2 \times 10^{14} < 2^{49}$. t1, t2, t3, R y el sumador/restador son de 50 bits.
- BCD: 32 bits binarios necesitan 10 dígitos decimales. El display muestra los 7 de abajo; si los dígitos 7 a 9 no son cero, LD1 (desborde) se enciende.

9. Camino de datos

El camino de datos se organiza alrededor de dos buses: la salida de la RAM (`dout_ram`) y la salida del sumador/restador (`resultado`). Los multiplexores de la figura los controla el estado del ASM (sección 8.2).

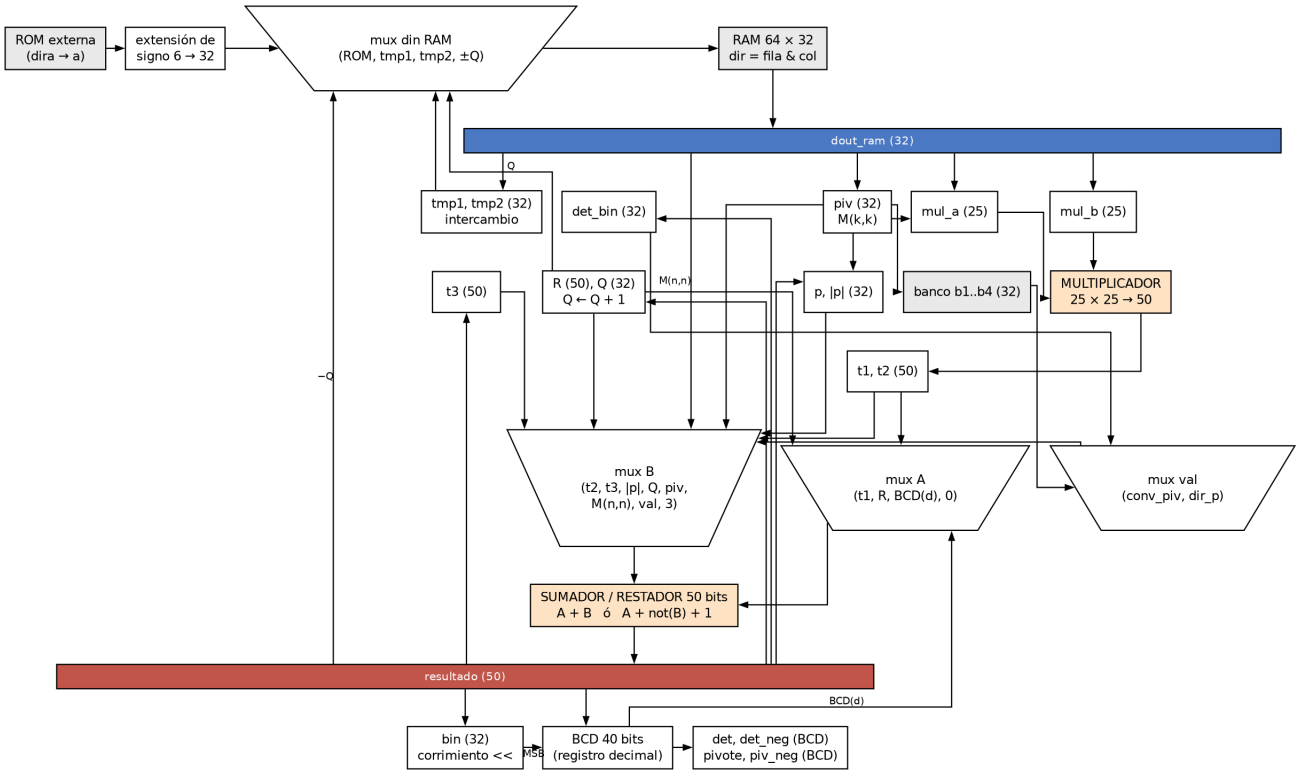


Figura 11. Diagrama de bloques simplificado del camino de datos.

Unidad	Detalle	Estados
Multiplicador	Entidad con signo, 25 × 25 → 50 bits: `p <= a * b;` con `std_logic_signed`.	s12, s14
Sumador/restador	Entidad de 50 bits: `s <= a + bx + op;` con bx = not b en resta. Un solo sumador con acarreo de entrada, como el bloque «+/−» de las notas de clase.	s15 a s21, s24
División por restas sucesivas	Registros R y Q, incrementador Q + 1 y el bit de signo de la resta R − |p| como señal de estado. Usa el sumador/restador.	s16 a s18
ROM	Externa al ASM: entidad `rom_matrices`, `with dir select`, 256 × 6 bits.	s1, s2
RAM	64 × 32 bits, escritura síncrona y lectura asíncrona; dirección fila & columna	s2 a s20
Banco de pivotes	4 registros de 32 bits; escritura en s10; lectura con dir_p (`with dir_p select`)	s3, s10, s21
Contadores	dira, i, j, k, Q, cont y d	varios
Comparadores	= 0, i = n, j = n, k + 1 = n, cont = 31, d = 9, dig > 4, dir_p ≠ dir_ant	varios

La síntesis genérica con GHDL (`ghdl --synth`) confirma un solo multiplicador (25 × 25), la RAM de 64 × 32 inferida como memoria y ningún latch.

10. Tiempo de ejecución

- Carga de la matriz (s0 a s3): $n^2 + 3$ ciclos.
- Paso k: s4, s10 y s19 (3 ciclos) más, por cada uno de los $(n - k)^2$ elementos, 8 ciclos fijos (s11 a s16 y s18) y |cociente| + 1 ciclos de división en s17.
- Intercambio de filas: hasta n − k ciclos de búsqueda, 4 ciclos por columna y una nueva lectura del pivote.
- XS3: unos 375 ciclos por conversión (s21 a s26). Al terminar el cálculo se hacen dos (det y pivote); al cambiar dir_p, una.

La división por restas sucesivas domina el tiempo cuando los valores son grandes, porque tarda tantos ciclos como vale el cociente. En la página 7 (5×5 con valores cercanos a ±32) el cálculo tarda unos 19 millones de ciclos, 0,19 s a 100 MHz. En el peor caso teórico el último cociente se acerca a $1{,}9 \times 10^9$, unos 19 s. Una división con corrimiento y resta (restauración) haría lo mismo en 50 ciclos; aquí se dejó la de restas sucesivas porque es la que pide el enunciado.

11. Implementación en ISE 14.7 y la Nexys A7

11.1 Proyecto

- File → New Project. Top-level source type: HDL.
- Family: Artix7 · Device: XC7A100T · Package: CSG324 · Speed: −1 · Synthesis: XST · Simulator: ISim · Preferred language: VHDL. ISE solo soporta la Nexys A7-100T; la variante 50T exige Vivado.
- Add Source: src/det_bareiss.vhd y src/nexys_a7.ucf (Implementation); sim/tb_asm_bareiss.vhd y sim/tb_det_bareiss.vhd (Simulation). Si ISE no elige det_bareiss como tope, usen «Set as Top Module».
- Synthesize → Implement Design → Generate Programming File. Revisen en el reporte de tiempos que se cumpla la restricción de 10 ns (TS_sys_clk).
- Programación: según el manual de la tarjeta, la Nexys A7 no funciona con Digilent Adept. Usen iMPACT de ISE o el Hardware Manager de Vivado (Lab Edition) con el archivo .bit.

11.2 Operación en la tarjeta

- Pulsar BTNU (rst).
- Elegir la matriz con SW15..SW13 (0 a 7, tabla 11.3) y el pivote con SW2..SW0 (1 a 4).
- Pulsar BTNC (inicio). Cuando termina, LD0 (final) se enciende.
- Con SW3 = 0 el display muestra el determinante; con SW3 = 1 muestra el pivote elegido con SW2..SW0. AN7 es el signo y AN6..AN0 los 7 dígitos.
- Al mover SW2..SW0, el ASM convierte el nuevo pivote (LD0 se apaga unos 4 µs).
- Si LD1 (desborde) está encendido, el valor mostrado tiene más de 7 dígitos y se ven los 7 de abajo.
- Para calcular otra matriz: cambiar SW15..SW13 y pulsar BTNC de nuevo.

11.3 Matrices de prueba en la ROM

Pág.	n	Matriz A	det	Pivotes	Qué prueba
0	2	7 -3 -3 5	26	7	Caso básico 2×2
1	3	2 1 3 1 4 2 3 2 5	3	2, 7	Ejemplo del documento

Pág.	n	Matriz A	det	Pivotes	Qué prueba
2	4	0 2 1 3 2 1 0 1 1 0 3 2 3 1 2 4	-22	2, 4, 13	a11 = 0: intercambio en k = 1
3	5	4 -2 1 3 0 -2 6 2 -1 5 1 2 -7 4 2 3 -1 4 5 -3 0 5 2 -3 8	362	4, 20, -170, -639	5×5 general, pivotes negativos
4	3	-2 4 1 4 -8 3 1 3 5	50	-2, -10	Pivote negativo; intercambio en k = 2
5	4	1 2 3 0 2 4 1 5 3 1 -2 4 0 5 4 -3	150	1, -5, 25	Pivote cero en k = 2 (p ≠ 1)
6	5	0 0 0 0 0 0 3 1 2 4 0 1 5 -2 1 0 2 -2 6 3 0 4 1 3 7	0	—	Columna 1 en cero: singular, det = 0
7	5	30 27 24 -29 -31 27 -21 28 29 -24 24 28 2 -28 -28 -29 29 -28 -27 28 -31 -24 -28 28 -30	-17 121 828	30, -1359, 22146, 277564	Valores extremos; desborde del display

Para cambiar o agregar matrices, editen la lista MATRICES en sim/modelo_referencia.py y ejecútenlo: imprime el det y los pivotes esperados y las líneas de la ROM (`"dato" when "dirección",`) listas para pegar en la entidad rom_matrices. Cada página empieza en la dirección 32 × página.

11.4 Pines (src/nexys_a7.ucf)

Señal	Pines
clk	E3 (100 MHz)
rst / inicio	M18 (BTNU) / N17 (BTNC)
dir_p(0..2)	J15 (SW0), L16 (SW1), M13 (SW2)
ver	R15 (SW3)
sw_mat(0..2)	U12 (SW13), U11 (SW14), V10 (SW15)
final / desborde	H17 (LD0) / K15 (LD1)
seg(0..6) = CA..CG, dp	T10, R10, K16, K13, P15, T11, L18; DP = H15
an(0..7)	J17, J18, T9, J14, P14, T14, K2, U13

12. Simulación

- **sim/tb_asm_bareiss.vhd** conecta la ROM externa al ASM, como en la caja negra. Para cada matriz pone dir_p = 1, pulsa inicio, espera final = 1 y compara det, det_neg y el pivote 1. Después recorre dir_p = 2 a n − 1 y compara cada pivote, y verifica que det no cambie. El genérico MAX_MATRIZ del testbench vale 6 por defecto (unos 0,2 ms simulados); con 7 incluye la matriz extrema, que tarda bastante.

- **sim/tb_det_bareiss.vhd** prueba el tope con el sincronizador, la ROM y el display: calcula la página 3 y lee los 8 dígitos decodificando an y seg durante un barrido completo (2^{18} ciclos), primero con SW3 = 0 (det) y luego con SW3 = 1 y dir_p = 3 (pivote 3).

Pág.	Tamaño	det esperado	det simulado	LD1	Ciclos	Resultado
0	2x2	' 0000026'	' 0000026'	0	792	OK
1	3x3	' 0000003'	' 0000003'	0	819	OK
2	4x4	'-0000022'	'-0000022'	0	1 004	OK
3	5x5	' 0000362'	' 0000362'	0	4 580	OK
4	3x3	' 0000050'	' 0000050'	0	897	OK
5	4x4	' 0000150'	' 0000150'	0	1 184	OK
6	5x5	' 0000000'	' 0000000'	0	780	OK
7	5x5	'-7121828'	'-7121828'	1	19 013 802	OK

Los ciclos van desde que se suelta inicio hasta final = 1 e incluyen las dos conversiones XS3. Todos los pivotes también coinciden, y el testbench del tope leyó « 0000362» y «-0000170» en los segmentos (ver sim/resultado_ghdl.txt). Para comprobar que los testbench detectan errores se probaron dos fallas en una copia del diseño: «+2» en lugar de «+3» en el XS3, y saltar la conversión del pivote en s26. En ambos casos el testbench reportó errores.

13. Limitaciones y verificación pendiente

- **No se ejecutó ISE.** El diseño se verificó con GHDL 4.1 (simulación con `-fsynopsys -fexplicit` y síntesis genérica), no con XST ni ISim. Hay que sintetizar en ISE 14.7 y revisar el reporte de tiempos a 100 MHz. Si no se cumpliera, generen un reloj de 50 MHz con el Clocking Wizard y pongan PERIOD = 20 ns.
- **Salidas det y pivote en BCD.** En la caja negra de los apuntes no se ve si Det y Pivote salen en binario o en BCD. Aquí salen en BCD porque el XS3 debe estar dentro del ASM. Si el profesor las quiere en binario, los valores ya existen dentro del ASM (det_bin y el banco).
- **Estados como constantes.** Si el profesor acepta `type estados_asm` como en sus ejemplos, basta cambiar la declaración de st y borrar las constantes; el resto del código no cambia.
- **Modelo de RAM.** Se usó lectura asíncrona. Si el modelo del aula tiene lectura síncrona, cada lectura (s4, s5, s6, s7, s11, s12, s13, s20) necesita un estado de espera antes de usar el dato.
- **Display de 7 dígitos.** Con signo en AN7 caben valores hasta ±9 999 999; para valores mayores se enciende LD1 y se ven los 7 dígitos de abajo.
- **Tiempo de la división.** Proporcional al cociente (sección 10).
- **Valor de n.** El diseño supone $2 \leq n \leq 5$ en la ROM y no lo valida.
- **ROM con 8 páginas.** La selección con SW15..SW13 permite mostrar varios tamaños sin volver a sintetizar. Si el profesor quiere una sola matriz, usen la página 0 con esos switches en 0.

Anexo A. Código VHDL completo (src/det_bareiss.vhd)

```
1  --------------------------------------------------------------------------------
2  -- Universidad Distrital Francisco Jose de Caldas
3  -- Analisis y Diseno de Microprocesadores - Taller 5
4  --
5  -- Microprocesador de proposito especifico descrito con un diagrama ASM
6  -- (modelo VHDL de alto nivel, como los ejemplos de clase):
7  --   * Determinante de una matriz simetrica de 2x2 a 5x5 por el algoritmo de
8  --     Bareiss (eliminacion libre de fracciones).
9  --   * Division por restas sucesivas.
10 --   * Conversion binario -> BCD con el algoritmo XS3 (sumar 3 y desplazar),
11 --     descrita paso a paso dentro del mismo ASM (estados s22 a s25).
12 --   * Banco de registros con los pivotes; se eligen con dir_p (switches).
13 --
14 -- Herramienta : Xilinx ISE 14.7 (XST / ISim)
15 -- Tarjeta     : Digilent Nexys A7-100T (XC7A100T-1CSG324C), reloj de 100 MHz
16 --
17 -- Reglas de descripcion:
18 --   * Solo senales de tipo std_logic y std_logic_vector: sin variables, sin
19 --     integer, sin signed/unsigned, sin tipos enumerados, sin genericos y sin
20 --     lazos for. Los estados del ASM son constantes std_logic_vector.
21 --   * Unica excepcion: la RAM, descrita como un arreglo de std_logic_vector
22 --     (es la forma en que XST infiere una memoria).
23 --   * El ASM completo esta en un solo proceso (entidad asm_bareiss), con
24 --     "case st is" y las operaciones de cada caja escritas en su estado.
25 --
26 -- Caja negra del procesador (entidad asm_bareiss), como en los apuntes:
27 --
28 --            +---------------------------+
29 --     clk -->|                           |--> det    (BCD) + det_neg
30 --     rst -->|    Algoritmo Bareiss      |--> pivote (BCD) + piv_neg
31 --  inicio -->|         + XS3             |--> final
32 --   dir_p -->|                           |--> dira ----+
33 --       a -->|                           |             |
34 --            +---------------------------+             v
35 --                  ^                              +---------------+
36 --                  +----------- a ------------|      ROM      |
37 --                                              +---------------+
38 --
39 -- Entidades de este archivo:
40 --   rom_matrices                      : ROM con las matrices (fuera del ASM)
41 --   ram_trabajo, multiplicador,
42 --   sumador_restador                  : bloques usados por el ASM
43 --   asm_bareiss                       : el diagrama ASM (s0 a s27)
44 --   sincronizador, display_8dig       : interfaz con la tarjeta
45 --   det_bareiss                       : tope para la Nexys A7
46 --------------------------------------------------------------------------------
47
48
49 --------------------------------------------------------------------------------
50 -- ROM DE MATRICES: 256 x 6 bits, lectura combinacional.
51 -- dir = pag(3) & dira(5). En cada pagina: 00h = n, luego los a(i,j) por columnas
52 -- (a11, a21, ..., an1, a12, ...), como en la Figura 1 del taller.
53 -- Datos en complemento a 2 (-32 a 31).
54 --------------------------------------------------------------------------------
55 library IEEE;
56 use IEEE.STD_LOGIC_1164.ALL;
57
58 entity rom_matrices is
59     Port ( dir : in  STD_LOGIC_VECTOR (7 downto 0);
60            a   : out STD_LOGIC_VECTOR (5 downto 0));
61 end rom_matrices;
62
63 architecture Behavioral of rom_matrices is
64 begin
65 with dir select
66     a <= "000010" when "00000000",   -- pag 0, 00h: n = 2
67          "000111" when "00000001",   -- pag 0, 01h: a11 = 7
68          "111101" when "00000010",   -- pag 0, 02h: a21 = -3
69          "111101" when "00000011",   -- pag 0, 03h: a12 = -3
70          "000101" when "00000100",   -- pag 0, 04h: a22 = 5
71          "000011" when "00100000",   -- pag 1, 00h: n = 3
72          "000010" when "00100001",   -- pag 1, 01h: a11 = 2
73          "000001" when "00100010",   -- pag 1, 02h: a21 = 1
74          "000011" when "00100011",   -- pag 1, 03h: a31 = 3
75          "000001" when "00100100",   -- pag 1, 04h: a12 = 1
76          "000100" when "00100101",   -- pag 1, 05h: a22 = 4
77          "000010" when "00100110",   -- pag 1, 06h: a32 = 2
78          "000011" when "00100111",   -- pag 1, 07h: a13 = 3
79          "000010" when "00101000",   -- pag 1, 08h: a23 = 2
80          "000101" when "00101001",   -- pag 1, 09h: a33 = 5
81          "000100" when "01000000",   -- pag 2, 00h: n = 4
82          "000000" when "01000001",   -- pag 2, 01h: a11 = 0
83          "000010" when "01000010",   -- pag 2, 02h: a21 = 2
84          "000001" when "01000011",   -- pag 2, 03h: a31 = 1
85          "000011" when "01000100",   -- pag 2, 04h: a41 = 3
```

```
          "000010" when "01000101",   -- pag 2, 05h: a12 = 2
          "000001" when "01000110",   -- pag 2, 06h: a22 = 1
          "000000" when "01000111",   -- pag 2, 07h: a32 = 0
          "000001" when "01001000",   -- pag 2, 08h: a42 = 1
          "000001" when "01001001",   -- pag 2, 09h: a13 = 1
          "000000" when "01001010",   -- pag 2, 0Ah: a23 = 0
          "000011" when "01001011",   -- pag 2, 0Bh: a33 = 3
          "000010" when "01001100",   -- pag 2, 0Ch: a43 = 2
          "000011" when "01001101",   -- pag 2, 0Dh: a14 = 3
          "000001" when "01001110",   -- pag 2, 0Eh: a24 = 1
          "000010" when "01001111",   -- pag 2, 0Fh: a34 = 2
          "000100" when "01010000",   -- pag 2, 10h: a44 = 4
          "000101" when "01100000",   -- pag 3, 00h: n = 5
          "000100" when "01100001",   -- pag 3, 01h: a11 = 4
          "111110" when "01100010",   -- pag 3, 02h: a21 = -2
          "000001" when "01100011",   -- pag 3, 03h: a31 = 1
          "000011" when "01100100",   -- pag 3, 04h: a41 = 3
          "000000" when "01100101",   -- pag 3, 05h: a51 = 0
          "111110" when "01100110",   -- pag 3, 06h: a12 = -2
          "000110" when "01100111",   -- pag 3, 07h: a22 = 6
          "000010" when "01101000",   -- pag 3, 08h: a32 = 2
          "111111" when "01101001",   -- pag 3, 09h: a42 = -1
          "000101" when "01101010",   -- pag 3, 0Ah: a52 = 5
          "000001" when "01101011",   -- pag 3, 0Bh: a13 = 1
          "000010" when "01101100",   -- pag 3, 0Ch: a23 = 2
          "111001" when "01101101",   -- pag 3, 0Dh: a33 = -7
          "000100" when "01101110",   -- pag 3, 0Eh: a43 = 4
          "000010" when "01101111",   -- pag 3, 0Fh: a53 = 2
          "000011" when "01110000",   -- pag 3, 10h: a14 = 3
          "111111" when "01110001",   -- pag 3, 11h: a24 = -1
          "000100" when "01110010",   -- pag 3, 12h: a34 = 4
          "000101" when "01110011",   -- pag 3, 13h: a44 = 5
          "111101" when "01110100",   -- pag 3, 14h: a54 = -3
          "000000" when "01110101",   -- pag 3, 15h: a15 = 0
          "000101" when "01110110",   -- pag 3, 16h: a25 = 5
          "000010" when "01110111",   -- pag 3, 17h: a35 = 2
          "111101" when "01111000",   -- pag 3, 18h: a45 = -3
          "001000" when "01111001",   -- pag 3, 19h: a55 = 8
          "000011" when "10000000",   -- pag 4, 00h: n = 3
          "111110" when "10000001",   -- pag 4, 01h: a11 = -2
          "000100" when "10000010",   -- pag 4, 02h: a21 = 4
          "000001" when "10000011",   -- pag 4, 03h: a31 = 1
          "000100" when "10000100",   -- pag 4, 04h: a12 = 4
          "111000" when "10000101",   -- pag 4, 05h: a22 = -8
          "000011" when "10000110",   -- pag 4, 06h: a32 = 3
          "000001" when "10000111",   -- pag 4, 07h: a13 = 1
          "000011" when "10001000",   -- pag 4, 08h: a23 = 3
          "000101" when "10001001",   -- pag 4, 09h: a33 = 5
          "000100" when "10100000",   -- pag 5, 00h: n = 4
          "000001" when "10100001",   -- pag 5, 01h: a11 = 1
          "000010" when "10100010",   -- pag 5, 02h: a21 = 2
          "000011" when "10100011",   -- pag 5, 03h: a31 = 3
          "000000" when "10100100",   -- pag 5, 04h: a41 = 0
          "000010" when "10100101",   -- pag 5, 05h: a12 = 2
          "000100" when "10100110",   -- pag 5, 06h: a22 = 4
          "000001" when "10100111",   -- pag 5, 07h: a32 = 1
          "000101" when "10101000",   -- pag 5, 08h: a42 = 5
          "000011" when "10101001",   -- pag 5, 09h: a13 = 3
          "000001" when "10101010",   -- pag 5, 0Ah: a23 = 1
          "111110" when "10101011",   -- pag 5, 0Bh: a33 = -2
          "000100" when "10101100",   -- pag 5, 0Ch: a43 = 4
          "000000" when "10101101",   -- pag 5, 0Dh: a14 = 0
          "000101" when "10101110",   -- pag 5, 0Eh: a24 = 5
          "000100" when "10101111",   -- pag 5, 0Fh: a34 = 4
          "111101" when "10110000",   -- pag 5, 10h: a44 = -3
          "000101" when "11000000",   -- pag 6, 00h: n = 5
          "000000" when "11000001",   -- pag 6, 01h: a11 = 0
          "000000" when "11000010",   -- pag 6, 02h: a21 = 0
          "000000" when "11000011",   -- pag 6, 03h: a31 = 0
          "000000" when "11000100",   -- pag 6, 04h: a41 = 0
          "000000" when "11000101",   -- pag 6, 05h: a51 = 0
          "000000" when "11000110",   -- pag 6, 06h: a12 = 0
          "000011" when "11000111",   -- pag 6, 07h: a22 = 3
          "000001" when "11001000",   -- pag 6, 08h: a32 = 1
          "000010" when "11001001",   -- pag 6, 09h: a42 = 2
          "000100" when "11001010",   -- pag 6, 0Ah: a52 = 4
          "000000" when "11001011",   -- pag 6, 0Bh: a13 = 0
          "000001" when "11001100",   -- pag 6, 0Ch: a23 = 1
          "000101" when "11001101",   -- pag 6, 0Dh: a33 = 5
          "111110" when "11001110",   -- pag 6, 0Eh: a43 = -2
          "000001" when "11001111",   -- pag 6, 0Fh: a53 = 1
          "000000" when "11010000",   -- pag 6, 10h: a14 = 0
          "000010" when "11010001",   -- pag 6, 11h: a24 = 2
          "111110" when "11010010",   -- pag 6, 12h: a34 = -2
          "000110" when "11010011",   -- pag 6, 13h: a44 = 6
          "000011" when "11010100",   -- pag 6, 14h: a54 = 3
          "000000" when "11010101",   -- pag 6, 15h: a15 = 0
          "000100" when "11010110",   -- pag 6, 16h: a25 = 4
```

```
          "000001" when "11010111",   -- pag 6, 17h: a35 = 1
          "000011" when "11011000",   -- pag 6, 18h: a45 = 3
          "000111" when "11011001",   -- pag 6, 19h: a55 = 7
          "000101" when "11100000",   -- pag 7, 00h: n = 5
          "011110" when "11100001",   -- pag 7, 01h: a11 = 30
          "011011" when "11100010",   -- pag 7, 02h: a21 = 27
          "011000" when "11100011",   -- pag 7, 03h: a31 = 24
          "100011" when "11100100",   -- pag 7, 04h: a41 = -29
          "100001" when "11100101",   -- pag 7, 05h: a51 = -31
          "011011" when "11100110",   -- pag 7, 06h: a12 = 27
          "101011" when "11100111",   -- pag 7, 07h: a22 = -21
          "011100" when "11101000",   -- pag 7, 08h: a32 = 28
          "011101" when "11101001",   -- pag 7, 09h: a42 = 29
          "101000" when "11101010",   -- pag 7, 0Ah: a52 = -24
          "011000" when "11101011",   -- pag 7, 0Bh: a13 = 24
          "011100" when "11101100",   -- pag 7, 0Ch: a23 = 28
          "000010" when "11101101",   -- pag 7, 0Dh: a33 = 2
          "100100" when "11101110",   -- pag 7, 0Eh: a43 = -28
          "100100" when "11101111",   -- pag 7, 0Fh: a53 = -28
          "100011" when "11110000",   -- pag 7, 10h: a14 = -29
          "011101" when "11110001",   -- pag 7, 11h: a24 = 29
          "100100" when "11110010",   -- pag 7, 12h: a34 = -28
          "100101" when "11110011",   -- pag 7, 13h: a44 = -27
          "011100" when "11110100",   -- pag 7, 14h: a54 = 28
          "100001" when "11110101",   -- pag 7, 15h: a15 = -31
          "101000" when "11110110",   -- pag 7, 16h: a25 = -24
          "100100" when "11110111",   -- pag 7, 17h: a35 = -28
          "011100" when "11111000",   -- pag 7, 18h: a45 = 28
          "100010" when "11111001",   -- pag 7, 19h: a55 = -30
          "000000" when others;
end Behavioral;


----------------------------------------------------------------------------------
-- RAM DE TRABAJO: 64 x 32 bits. Escritura sincrona, lectura asincrona.
-- dir = fila(3) & columna(3).
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity ram_trabajo is
    Port ( clk  : in  STD_LOGIC;
           we   : in  STD_LOGIC;
           dir  : in  STD_LOGIC_VECTOR (5 downto 0);
           din  : in  STD_LOGIC_VECTOR (31 downto 0);
           dout : out STD_LOGIC_VECTOR (31 downto 0));
end ram_trabajo;

architecture Behavioral of ram_trabajo is
type ram_tipo is array (0 to 63) of std_logic_vector (31 downto 0);
signal ram : ram_tipo := (others => (others => '0'));
begin

process (clk)
begin
    if clk'event and clk = '1' then
        if we = '1' then
            ram(conv_integer(dir)) <= din;
        end if;
    end if;
end process;

dout <= ram(conv_integer(dir));

end Behavioral;


----------------------------------------------------------------------------------
-- MULTIPLICADOR CON SIGNO 25 x 25 -> 50 bits (combinacional).
-- std_logic_signed interpreta los std_logic_vector en complemento a 2.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_signed.all;

entity multiplicador is
    Port ( a : in  STD_LOGIC_VECTOR (24 downto 0);
           b : in  STD_LOGIC_VECTOR (24 downto 0);
           p : out STD_LOGIC_VECTOR (49 downto 0));
end multiplicador;

architecture Behavioral of multiplicador is
begin
p <= a * b;
end Behavioral;


```

```
----------------------------------------------------------------------------------
-- SUMADOR/RESTADOR DE 50 bits (combinacional).
-- op = '0': s = a + b      op = '1': s = a + not(b) + 1 = a - b
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity sumador_restador is
    Port ( a  : in  STD_LOGIC_VECTOR (49 downto 0);
           b  : in  STD_LOGIC_VECTOR (49 downto 0);
           op : in  STD_LOGIC;
           s  : out STD_LOGIC_VECTOR (49 downto 0));
end sumador_restador;

architecture Behavioral of sumador_restador is
signal bx : STD_LOGIC_VECTOR (49 downto 0);
begin
bx <= not b when op = '1' else b;
s  <= a + bx + op;
end Behavioral;


----------------------------------------------------------------------------------
-- ASM DEL PROCESADOR: Bareiss + division por restas sucesivas + XS3
-- El proceso del final es el diagrama ASM: un "when" por estado (s0 a s27),
-- las cajas de decision son "if" y las cajas de salida condicional son las
-- asignaciones dentro del "if".
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity asm_bareiss is
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);   -- direccion del banco de pivotes (1..4)
           a        : in  STD_LOGIC_VECTOR (5 downto 0);   -- dato de la ROM
           dira     : out STD_LOGIC_VECTOR (4 downto 0);   -- direccion de la ROM
           det      : out STD_LOGIC_VECTOR (39 downto 0);  -- |det| en BCD (10 digitos)
           det_neg  : out STD_LOGIC;                       -- signo del determinante
           pivote   : out STD_LOGIC_VECTOR (39 downto 0);  -- |pivote(dir_p)| en BCD
           piv_neg  : out STD_LOGIC;                       -- signo del pivote
           final    : out STD_LOGIC);
end asm_bareiss;

architecture Behavioral of asm_bareiss is

component ram_trabajo
    Port ( clk  : in  STD_LOGIC;
           we   : in  STD_LOGIC;
           dir  : in  STD_LOGIC_VECTOR (5 downto 0);
           din  : in  STD_LOGIC_VECTOR (31 downto 0);
           dout : out STD_LOGIC_VECTOR (31 downto 0));
end component;

component multiplicador
    Port ( a : in  STD_LOGIC_VECTOR (24 downto 0);
           b : in  STD_LOGIC_VECTOR (24 downto 0);
           p : out STD_LOGIC_VECTOR (49 downto 0));
end component;

component sumador_restador
    Port ( a  : in  STD_LOGIC_VECTOR (49 downto 0);
           b  : in  STD_LOGIC_VECTOR (49 downto 0);
           op : in  STD_LOGIC;
           s  : out STD_LOGIC_VECTOR (49 downto 0));
end component;

-------------------------------------------------------------------------------
-- Estados del ASM (constantes std_logic_vector)
-------------------------------------------------------------------------------
-- s0        espera de inicio
-- s1..s2    lectura de n y copia de la matriz ROM -> RAM
-- s3        inicializacion de Bareiss
-- s4..s10   pivote: lectura, busqueda de fila, intercambio, banco de pivotes
-- s11..s18  actualizacion de a(i,j): multiplicaciones, resta y division
-- s19       actualizacion del pivote anterior p y avance de k
-- s20       det = (+/-) a(n,n)
-- s21       valor a convertir: det (1a pasada) o pivote(dir_p) (2a pasada)
-- s22..s25  algoritmo XS3 (pasos 1 a 5)
-- s26       guarda el BCD en det o en pivote
-- s27       final = 1
constant s0  : STD_LOGIC_VECTOR (4 downto 0) := "00000";
constant s1  : STD_LOGIC_VECTOR (4 downto 0) := "00001";
constant s2  : STD_LOGIC_VECTOR (4 downto 0) := "00010";
constant s3  : STD_LOGIC_VECTOR (4 downto 0) := "00011";
constant s4  : STD_LOGIC_VECTOR (4 downto 0) := "00100";
```

```
constant s5  : STD_LOGIC_VECTOR (4 downto 0) := "00101";
constant s6  : STD_LOGIC_VECTOR (4 downto 0) := "00110";
constant s7  : STD_LOGIC_VECTOR (4 downto 0) := "00111";
constant s8  : STD_LOGIC_VECTOR (4 downto 0) := "01000";
constant s9  : STD_LOGIC_VECTOR (4 downto 0) := "01001";
constant s10 : STD_LOGIC_VECTOR (4 downto 0) := "01010";
constant s11 : STD_LOGIC_VECTOR (4 downto 0) := "01011";
constant s12 : STD_LOGIC_VECTOR (4 downto 0) := "01100";
constant s13 : STD_LOGIC_VECTOR (4 downto 0) := "01101";
constant s14 : STD_LOGIC_VECTOR (4 downto 0) := "01110";
constant s15 : STD_LOGIC_VECTOR (4 downto 0) := "01111";
constant s16 : STD_LOGIC_VECTOR (4 downto 0) := "10000";
constant s17 : STD_LOGIC_VECTOR (4 downto 0) := "10001";
constant s18 : STD_LOGIC_VECTOR (4 downto 0) := "10010";
constant s19 : STD_LOGIC_VECTOR (4 downto 0) := "10011";
constant s20 : STD_LOGIC_VECTOR (4 downto 0) := "10100";
constant s21 : STD_LOGIC_VECTOR (4 downto 0) := "10101";
constant s22 : STD_LOGIC_VECTOR (4 downto 0) := "10110";
constant s23 : STD_LOGIC_VECTOR (4 downto 0) := "10111";
constant s24 : STD_LOGIC_VECTOR (4 downto 0) := "11000";
constant s25 : STD_LOGIC_VECTOR (4 downto 0) := "11001";
constant s26 : STD_LOGIC_VECTOR (4 downto 0) := "11010";
constant s27 : STD_LOGIC_VECTOR (4 downto 0) := "11011";

signal st : STD_LOGIC_VECTOR (4 downto 0);

-------------------------------------------------------------------------------
-- Constantes
-------------------------------------------------------------------------------
constant CERO18 : STD_LOGIC_VECTOR (17 downto 0) := "000000000000000000";
constant CERO32 : STD_LOGIC_VECTOR (31 downto 0) := x"00000000";
constant UNO32  : STD_LOGIC_VECTOR (31 downto 0) := x"00000001";
constant CERO40 : STD_LOGIC_VECTOR (39 downto 0) := x"0000000000";
constant CERO46 : STD_LOGIC_VECTOR (45 downto 0) := (others => '0');
constant CERO50 : STD_LOGIC_VECTOR (49 downto 0) := (others => '0');
constant TRES50 : STD_LOGIC_VECTOR (49 downto 0) := (1 downto 0 => '1', others => '0');

-------------------------------------------------------------------------------
-- ROM (externa) y RAM
-------------------------------------------------------------------------------
signal dira_r   : STD_LOGIC_VECTOR (4 downto 0);   -- registro de direccion de la ROM
signal ext_a    : STD_LOGIC_VECTOR (25 downto 0);
signal a32      : STD_LOGIC_VECTOR (31 downto 0);  -- dato de la ROM a 32 bits
signal fila_ram, col_ram : STD_LOGIC_VECTOR (2 downto 0);
signal dir_ram  : STD_LOGIC_VECTOR (5 downto 0);
signal din_ram  : STD_LOGIC_VECTOR (31 downto 0);
signal dout_ram : STD_LOGIC_VECTOR (31 downto 0);
signal we_ram   : STD_LOGIC;

-------------------------------------------------------------------------------
-- Registros del algoritmo de Bareiss
-------------------------------------------------------------------------------
signal n, i, j, k : STD_LOGIC_VECTOR (2 downto 0); -- tamano y contadores
signal piv   : STD_LOGIC_VECTOR (31 downto 0);     -- pivote actual a(k,k)
signal p     : STD_LOGIC_VECTOR (31 downto 0);     -- pivote anterior (divisor)
signal pabs  : STD_LOGIC_VECTOR (31 downto 0);     -- |p|
signal signo : STD_LOGIC;                          -- '1' = numero impar de intercambios
signal b1, b2, b3, b4 : STD_LOGIC_VECTOR (31 downto 0); -- banco de pivotes
signal tmp1, tmp2 : STD_LOGIC_VECTOR (31 downto 0); -- intercambio de filas
signal t1, t2, t3 : STD_LOGIC_VECTOR (49 downto 0); -- productos y numerador
signal det_bin : STD_LOGIC_VECTOR (31 downto 0);   -- determinante en binario

-- Multiplicador: producto = mul_a * mul_b
signal mul_a, mul_b : STD_LOGIC_VECTOR (24 downto 0);
signal producto     : STD_LOGIC_VECTOR (49 downto 0);

-- Sumador/restador: resultado = sr_a + sr_b  o  sr_a - sr_b
signal sr_a, sr_b, resultado : STD_LOGIC_VECTOR (49 downto 0);
signal sr_op : STD_LOGIC;                          -- '0' suma, '1' resta
signal sx_piv, sx_dout, sx_val : STD_LOGIC_VECTOR (17 downto 0);

-- Division por restas sucesivas
signal R  : STD_LOGIC_VECTOR (49 downto 0);        -- residuo
signal Q  : STD_LOGIC_VECTOR (31 downto 0);        -- cociente
signal sq : STD_LOGIC;                             -- signo del cociente

-------------------------------------------------------------------------------
-- Conversion XS3 (binario -> BCD)
-------------------------------------------------------------------------------
signal conv_piv : STD_LOGIC;                       -- '0' convierte det, '1' convierte pivote
signal piv_sel  : STD_LOGIC_VECTOR (31 downto 0);  -- banco(dir_p)
signal val      : STD_LOGIC_VECTOR (31 downto 0);  -- valor que se va a convertir
signal dir_ant  : STD_LOGIC_VECTOR (2 downto 0);   -- dir_p ya convertido
signal bin      : STD_LOGIC_VECTOR (31 downto 0);  -- registro binario
signal bcd      : STD_LOGIC_VECTOR (39 downto 0);  -- registro decimal: 10 digitos de 4 bits
signal cont     : STD_LOGIC_VECTOR (4 downto 0);   -- bits procesados (0..31)
signal d        : STD_LOGIC_VECTOR (3 downto 0);   -- digito revisado (0..9)
signal dig      : STD_LOGIC_VECTOR (3 downto 0);   -- digito d del registro decimal
```

```
signal neg      : STD_LOGIC;                      -- signo del valor convertido
signal det_bcd, piv_bcd : STD_LOGIC_VECTOR (39 downto 0);   -- resultados
signal det_neg_r, piv_neg_r : STD_LOGIC;

begin

-------------------------------------------------------------------------------
-- CAMINO DE DATOS: bloques y conexiones que dependen del estado
-------------------------------------------------------------------------------

-- ROM externa
dira  <= dira_r;
ext_a <= (others => a(5));
a32   <= ext_a & a;                                 -- extension de signo

-- RAM: direccion, dato y escritura segun el estado
fila_ram <= k when (st = s4 or st = s6 or st = s8 or st = s13) else
            n when st = s20 else
            i;
col_ram  <= k when (st = s4 or st = s5 or st = s12) else
            n when st = s20 else
            j;
dir_ram  <= fila_ram & col_ram;

we_ram   <= '1' when (st = s2 or st = s8 or st = s9 or st = s18) else '0';

din_ram  <= a32                    when st = s2 else    -- copia de la ROM
            tmp2                   when st = s8 else    -- intercambio
            tmp1                   when st = s9 else    -- intercambio
            resultado(31 downto 0) when sq = '1' else   -- s18: -Q
            Q;                                          -- s18: +Q

U_RAM : ram_trabajo port map (clk => clk, we => we_ram, dir => dir_ram,
                              din => din_ram, dout => dout_ram);

-- Multiplicador
U_MUL : multiplicador port map (a => mul_a, b => mul_b, p => producto);

-- Sumador/restador: operandos segun el estado
sx_piv  <= (others => piv(31));
sx_dout <= (others => dout_ram(31));
sx_val  <= (others => val(31));

sr_a <= t1           when st = s15 else                 -- t1 - t2
        R            when st = s17 else                 -- R - |p|
        CERO46 & dig when st = s24 else                 -- digito + 3
        CERO50;                                         -- 0 - x (negacion)
sr_b <= t2                  when st = s15 else
        t3                  when st = s16 else          -- -t3
        CERO18 & pabs       when st = s17 else          -- |p|
        CERO18 & Q          when st = s18 else          -- -Q
        sx_piv & piv        when st = s19 else          -- -piv
        sx_dout & dout_ram when st = s20 else           -- -a(n,n)
        sx_val & val        when st = s21 else          -- -valor
        TRES50              when st = s24 else          -- +3 (XS3)
        CERO50;
sr_op <= '0' when st = s24 else '1';

U_SR : sumador_restador port map (a => sr_a, b => sr_b, op => sr_op, s => resultado);

-- Banco de pivotes: lectura con dir_p
with dir_p select
    piv_sel <= b1     when "001",
               b2     when "010",
               b3     when "011",
               b4     when "100",
               CERO32 when others;

-- Valor a convertir: det en la primera pasada, pivote(dir_p) en la segunda
val <= det_bin when conv_piv = '0' else piv_sel;

-- XS3: digito d del registro decimal
with d select
    dig <= bcd(3 downto 0)   when "0000",
           bcd(7 downto 4)   when "0001",
           bcd(11 downto 8)  when "0010",
           bcd(15 downto 12) when "0011",
           bcd(19 downto 16) when "0100",
           bcd(23 downto 20) when "0101",
           bcd(27 downto 24) when "0110",
           bcd(31 downto 28) when "0111",
           bcd(35 downto 32) when "1000",
           bcd(39 downto 36) when "1001",
           "0000"            when others;

-- Salidas de la caja negra
final   <= '1' when st = s27 else '0';
det     <= det_bcd;
```

```
526  det_neg <= det_neg_r;
527  pivote  <= piv_bcd;
528  piv_neg <= piv_neg_r;
529
530  ------------------------------------------------------------------------------
531  -- DIAGRAMA ASM
532  ------------------------------------------------------------------------------
533  process (clk, rst)
534  begin
535
536  if rst = '1' then
537      st      <= s0;
538      dira_r  <= "00000";
539      n       <= "000";   i <= "000";   j <= "000";   k <= "000";
540      piv     <= CERO32;  p <= CERO32;  pabs <= CERO32;  signo <= '0';
541      b1      <= CERO32;  b2 <= CERO32;  b3 <= CERO32;  b4 <= CERO32;
542      tmp1    <= CERO32;  tmp2 <= CERO32;
543      mul_a   <= (others => '0');  mul_b <= (others => '0');
544      t1      <= CERO50;  t2 <= CERO50;  t3 <= CERO50;
545      R       <= CERO50;  Q <= CERO32;  sq <= '0';
546      det_bin <= CERO32;
547      conv_piv <= '0';    dir_ant <= "000";
548      bin     <= CERO32;  bcd <= CERO40;  cont <= "00000";  d <= "0000";  neg <= '0';
549      det_bcd <= CERO40;  det_neg_r <= '0';
550      piv_bcd <= CERO40;  piv_neg_r <= '0';
551
552  elsif clk'event and clk = '1' then
553
554      case st is
555
556          ---------------------------------------------------------------
557          -- Espera y carga de la matriz
558          ---------------------------------------------------------------
559          when s0 =>                              -- final = 0 ; dira = 0
560              dira_r <= "00000";
561              if inicio = '0' then
562                  st <= s0;
563              else
564                  st <= s1;
565              end if;
566
567          when s1 =>                              -- n = ROM(00h)
568              n      <= a(2 downto 0);
569              dira_r <= "00001";
570              i      <= "001";
571              j      <= "001";
572              st     <= s2;
573
574          when s2 =>                              -- M(i,j) = ROM(dira), por columnas
575              dira_r <= dira_r + '1';
576              if i = n then
577                  i <= "001";
578                  if j = n then
579                      st <= s3;
580                  else
581                      j  <= j + '1';
582                      st <= s2;
583                  end if;
584              else
585                  i  <= i + '1';
586                  st <= s2;
587              end if;
588
589          when s3 =>                              -- inicializacion de Bareiss
590              k        <= "001";
591              p        <= UNO32;                  -- pivote inicial a(0,0) = 1
592              pabs     <= UNO32;
593              signo    <= '0';
594              b1 <= CERO32;  b2 <= CERO32;  b3 <= CERO32;  b4 <= CERO32;
595              conv_piv <= '0';
596              st       <= s4;
597
598          ---------------------------------------------------------------
599          -- Pivote del paso k
600          ---------------------------------------------------------------
601          when s4 =>                              -- piv = M(k,k)
602              piv <= dout_ram;
603              if dout_ram = CERO32 then
604                  i  <= k + '1';
605                  st <= s5;
606              else
607                  st <= s10;
608              end if;
609
610          when s5 =>                              -- busca fila i > k con M(i,k) /= 0
611              if dout_ram /= CERO32 then
612                  j     <= k;
613                  signo <= not signo;             -- el intercambio cambia el signo
```

```
614                 st    <= s6;
615             elsif i = n then
616                 det_bin <= CERO32;              -- no hay fila: matriz singular
617                 st      <= s21;
618             else
619                 i  <= i + '1';
620                 st <= s5;
621             end if;
622
623         when s6 =>                              -- tmp1 = M(k,j)
624             tmp1 <= dout_ram;
625             st   <= s7;
626
627         when s7 =>                              -- tmp2 = M(i,j)
628             tmp2 <= dout_ram;
629             st   <= s8;
630
631         when s8 =>                              -- M(k,j) = tmp2
632             st <= s9;
633
634         when s9 =>                              -- M(i,j) = tmp1
635             if j = n then
636                 st <= s4;                       -- vuelve a leer el pivote
637             else
638                 j  <= j + '1';
639                 st <= s6;
640             end if;
641
642         when s10 =>                             -- banco(k) = piv ; i = j = k + 1
643             case k is
644                 when "001"  => b1 <= piv;
645                 when "010"  => b2 <= piv;
646                 when "011"  => b3 <= piv;
647                 when "100"  => b4 <= piv;
648                 when others => null;
649             end case;
650             i  <= k + '1';
651             j  <= k + '1';
652             st <= s11;
653
654         ------------------------------------------------------------------
655         -- a(i,j) = ( a(k,k)*a(i,j) - a(i,k)*a(k,j) ) / p
656         ------------------------------------------------------------------
657         when s11 =>                             -- mul_a = a(k,k) ; mul_b = M(i,j)
658             mul_a <= piv(24 downto 0);
659             mul_b <= dout_ram(24 downto 0);
660             st    <= s12;
661
662         when s12 =>                             -- t1 = mul_a * mul_b ; mul_a = M(i,k)
663             t1    <= producto;
664             mul_a <= dout_ram(24 downto 0);
665             st    <= s13;
666
667         when s13 =>                             -- mul_b = M(k,j)
668             mul_b <= dout_ram(24 downto 0);
669             st    <= s14;
670
671         when s14 =>                             -- t2 = mul_a * mul_b
672             t2 <= producto;
673             st <= s15;
674
675         when s15 =>                             -- t3 = t1 - t2
676             t3 <= resultado;
677             st <= s16;
678
679         when s16 =>                             -- R = |t3| ; Q = 0 ; signo del cociente
680             Q  <= CERO32;
681             sq <= t3(49) xor p(31);
682             if t3(49) = '1' then
683                 R <= resultado;                 -- R = 0 - t3
684             else
685                 R <= t3;
686             end if;
687             st <= s17;
688
689         when s17 =>                             -- division por restas sucesivas
690             if resultado(49) = '0' then         -- R - |p| >= 0
691                 R  <= resultado;                -- R = R - |p|
692                 Q  <= Q + '1';
693                 st <= s17;
694             else
695                 st <= s18;
696             end if;
697
698         when s18 =>                             -- M(i,j) = Q o -Q ; avanza j, i
699             if j = n then
700                 if i = n then
701                     st <= s19;
```

```
                else
                    i  <= i + '1';
                    j  <= k + '1';
                    st <= s11;
                end if;
            else
                j  <= j + '1';
                st <= s11;
            end if;

        when s19 =>                             -- p = piv ; |p| = |piv| ; k = k + 1
            p <= piv;
            if piv(31) = '1' then
                pabs <= resultado(31 downto 0); -- |p| = 0 - piv
            else
                pabs <= piv;
            end if;
            k <= k + '1';
            if k + '1' = n then                 -- k = n - 1 : ultimo paso
                st <= s20;
            else
                st <= s4;
            end if;

        when s20 =>                             -- det = M(n,n) o -M(n,n)
            if signo = '1' then
                det_bin <= resultado(31 downto 0);  -- det = 0 - M(n,n)
            else
                det_bin <= dout_ram;
            end if;
            st <= s21;

        ----------------------------------------------------------------
        -- Conversion XS3 de det (conv_piv = 0) y del pivote (conv_piv = 1)
        ----------------------------------------------------------------
        when s21 =>                             -- bin = |val| ; neg = signo de val
            dir_ant <= dir_p;
            neg     <= val(31);
            if val(31) = '1' then
                bin <= resultado(31 downto 0);  -- bin = 0 - val
            else
                bin <= val;
            end if;
            st <= s22;

        when s22 =>                             -- XS3 paso 1: registro decimal = 0
            bcd  <= CERO40;
            cont <= "00000";
            st   <= s23;

        when s23 =>                             -- XS3 paso 2: LSB decimal = MSB binario
            bcd(0) <= bin(31);
            if cont = "11111" then
                st <= s26;                      -- paso 5: ultimo paso 2 hecho
            else
                d  <= "0000";
                st <= s24;
            end if;

        when s24 =>                             -- XS3 paso 3: si digito(d) > 4, sumar 3
            if dig > "0100" then
                case d is                       -- digito(d) = digito(d) + 3
                    when "0000" => bcd(3 downto 0)   <= resultado(3 downto 0);
                    when "0001" => bcd(7 downto 4)   <= resultado(3 downto 0);
                    when "0010" => bcd(11 downto 8)  <= resultado(3 downto 0);
                    when "0011" => bcd(15 downto 12) <= resultado(3 downto 0);
                    when "0100" => bcd(19 downto 16) <= resultado(3 downto 0);
                    when "0101" => bcd(23 downto 20) <= resultado(3 downto 0);
                    when "0110" => bcd(27 downto 24) <= resultado(3 downto 0);
                    when "0111" => bcd(31 downto 28) <= resultado(3 downto 0);
                    when "1000" => bcd(35 downto 32) <= resultado(3 downto 0);
                    when "1001" => bcd(39 downto 36) <= resultado(3 downto 0);
                    when others => null;
                end case;
            end if;
            if d = "1001" then
                st <= s25;
            else
                d  <= d + '1';
                st <= s24;
            end if;

        when s25 =>                             -- XS3 paso 4: corrimiento a la izquierda
            bcd  <= bcd(38 downto 0) & '0';
            bin  <= bin(30 downto 0) & '0';
            cont <= cont + '1';
            st   <= s23;

```

```
        when s26 =>                             -- guarda el resultado de la conversion
            if conv_piv = '0' then
                det_bcd   <= bcd;               -- det convertido
                det_neg_r <= neg;
                conv_piv  <= '1';
                st        <= s21;               -- ahora convierte el pivote
            else
                piv_bcd   <= bcd;               -- pivote convertido
                piv_neg_r <= neg;
                st        <= s27;
            end if;

        when s27 =>                             -- final = 1
            dira_r <= "00000";
            if inicio = '1' then
                st <= s1;                       -- nuevo calculo
            elsif dir_p /= dir_ant then
                st <= s21;                      -- cambio dir_p: convierte el nuevo pivote
            else
                st <= s27;
            end if;

        when others =>
            st <= s0;

    end case;
end if;
end process;

end Behavioral;


----------------------------------------------------------------------------------
-- SINCRONIZADOR: dos flip-flops para el boton y los switches
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sincronizador is
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);
           dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);
           ver      : in  STD_LOGIC;
           inicio_s : out STD_LOGIC;
           sw_mat_s : out STD_LOGIC_VECTOR (2 downto 0);
           dir_p_s  : out STD_LOGIC_VECTOR (2 downto 0);
           ver_s    : out STD_LOGIC);
end sincronizador;

architecture Behavioral of sincronizador is
signal ini_1, ini_2, ver_1, ver_2 : STD_LOGIC;
signal mat_1, mat_2, dir_1, dir_2 : STD_LOGIC_VECTOR (2 downto 0);
begin

process (clk, rst)
begin
    if rst = '1' then
        ini_1 <= '0';    ini_2 <= '0';
        ver_1 <= '0';    ver_2 <= '0';
        mat_1 <= "000";  mat_2 <= "000";
        dir_1 <= "000";  dir_2 <= "000";
    elsif clk'event and clk = '1' then
        ini_1 <= inicio;  ini_2 <= ini_1;
        ver_1 <= ver;     ver_2 <= ver_1;
        mat_1 <= sw_mat;  mat_2 <= mat_1;
        dir_1 <= dir_p;   dir_2 <= dir_1;
    end if;
end process;

inicio_s <= ini_2;
ver_s    <= ver_2;
sw_mat_s <= mat_2;
dir_p_s  <= dir_2;

end Behavioral;


----------------------------------------------------------------------------------
-- DISPLAY DE 8 DIGITOS MULTIPLEXADO (Nexys A7, anodo comun, activos en bajo)
-- AN7 = signo ('-' o apagado), AN6..AN0 = 7 digitos BCD.
-- Contador de refresco de 18 bits: 2^15 ciclos por digito, 2,6 ms por barrido.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity display_8dig is
```

```
    Port ( clk, rst : in  STD_LOGIC;
           bcd      : in  STD_LOGIC_VECTOR (27 downto 0);
           neg      : in  STD_LOGIC;
           an       : out STD_LOGIC_VECTOR (7 downto 0);
           seg      : out STD_LOGIC_VECTOR (6 downto 0);   -- seg(0)=CA ... seg(6)=CG
           dp       : out STD_LOGIC);
end display_8dig;

architecture Behavioral of display_8dig is
signal refresco  : STD_LOGIC_VECTOR (17 downto 0);
signal dsel      : STD_LOGIC_VECTOR (2 downto 0);
signal car       : STD_LOGIC_VECTOR (3 downto 0);   -- 0-9, A = '-', F = apagado
signal car_signo : STD_LOGIC_VECTOR (3 downto 0);
begin

process (clk, rst)
begin
    if rst = '1' then
        refresco <= (others => '0');
    elsif clk'event and clk = '1' then
        refresco <= refresco + '1';
    end if;
end process;

dsel      <= refresco(17 downto 15);
car_signo <= "1010" when neg = '1' else "1111";

with dsel select
    car <= bcd(3 downto 0)   when "000",
           bcd(7 downto 4)   when "001",
           bcd(11 downto 8)  when "010",
           bcd(15 downto 12) when "011",
           bcd(19 downto 16) when "100",
           bcd(23 downto 20) when "101",
           bcd(27 downto 24) when "110",
           car_signo         when others;

with dsel select
    an <= "11111110" when "000",
          "11111101" when "001",
          "11111011" when "010",
          "11110111" when "011",
          "11101111" when "100",
          "11011111" when "101",
          "10111111" when "110",
          "01111111" when others;

with car select                                 --  gfedcba
    seg <= "1000000" when "0000",               --  0
           "1111001" when "0001",               --  1
           "0100100" when "0010",               --  2
           "0110000" when "0011",               --  3
           "0011001" when "0100",               --  4
           "0010010" when "0101",               --  5
           "0000010" when "0110",               --  6
           "1111000" when "0111",               --  7
           "0000000" when "1000",               --  8
           "0010000" when "1001",               --  9
           "0111111" when "1010",               --  -
           "1111111" when others;               --  apagado

dp <= '1';

end Behavioral;


----------------------------------------------------------------------------------
-- TOPE: det_bareiss (sincronizador + ROM + ASM + display)
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity det_bareiss is
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;                      -- BTNC
           sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);  -- SW15..SW13: matriz (pagina de la ROM)
           dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);  -- SW2..SW0 : direccion del pivote (1..4)
           ver      : in  STD_LOGIC;                      -- SW3: 0 = muestra det, 1 = muestra pivote
           final    : out STD_LOGIC;                      -- LD0
           desborde : out STD_LOGIC;                      -- LD1: el valor mostrado tiene mas de 7 digitos
           an       : out STD_LOGIC_VECTOR (7 downto 0);
           seg      : out STD_LOGIC_VECTOR (6 downto 0);
           dp       : out STD_LOGIC);
end det_bareiss;

architecture Behavioral of det_bareiss is

component sincronizador
```

```
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);
           dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);
           ver      : in  STD_LOGIC;
           inicio_s : out STD_LOGIC;
           sw_mat_s : out STD_LOGIC_VECTOR (2 downto 0);
           dir_p_s  : out STD_LOGIC_VECTOR (2 downto 0);
           ver_s    : out STD_LOGIC);
end component;

component rom_matrices
    Port ( dir : in  STD_LOGIC_VECTOR (7 downto 0);
           a   : out STD_LOGIC_VECTOR (5 downto 0));
end component;

component asm_bareiss
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);
           a        : in  STD_LOGIC_VECTOR (5 downto 0);
           dira     : out STD_LOGIC_VECTOR (4 downto 0);
           det      : out STD_LOGIC_VECTOR (39 downto 0);
           det_neg  : out STD_LOGIC;
           pivote   : out STD_LOGIC_VECTOR (39 downto 0);
           piv_neg  : out STD_LOGIC;
           final    : out STD_LOGIC);
end component;

component display_8dig
    Port ( clk, rst : in  STD_LOGIC;
           bcd      : in  STD_LOGIC_VECTOR (27 downto 0);
           neg      : in  STD_LOGIC;
           an       : out STD_LOGIC_VECTOR (7 downto 0);
           seg      : out STD_LOGIC_VECTOR (6 downto 0);
           dp       : out STD_LOGIC);
end component;

constant CERO12 : STD_LOGIC_VECTOR (11 downto 0) := x"000";

signal inicio_s, ver_s : STD_LOGIC;
signal sw_mat_s, dir_p_s : STD_LOGIC_VECTOR (2 downto 0);
signal dira     : STD_LOGIC_VECTOR (4 downto 0);
signal dir_rom  : STD_LOGIC_VECTOR (7 downto 0);
signal a        : STD_LOGIC_VECTOR (5 downto 0);
signal det, pivote, mostrar : STD_LOGIC_VECTOR (39 downto 0);
signal det_neg, piv_neg, mostrar_neg : STD_LOGIC;

begin

U_SINC : sincronizador port map (
    clk => clk, rst => rst, inicio => inicio, sw_mat => sw_mat, dir_p => dir_p, ver => ver,
    inicio_s => inicio_s, sw_mat_s => sw_mat_s, dir_p_s => dir_p_s, ver_s => ver_s);

-- ROM externa al ASM; la pagina la eligen los switches SW15..SW13
dir_rom <= sw_mat_s & dira;
U_ROM : rom_matrices port map (dir => dir_rom, a => a);

U_ASM : asm_bareiss port map (
    clk => clk, rst => rst, inicio => inicio_s, dir_p => dir_p_s, a => a, dira => dira,
    det => det, det_neg => det_neg, pivote => pivote, piv_neg => piv_neg, final => final);

-- Valor que se muestra en los displays
mostrar     <= det     when ver_s = '0' else pivote;
mostrar_neg <= det_neg when ver_s = '0' else piv_neg;
desborde    <= '0' when mostrar(39 downto 28) = CERO12 else '1';

U_DISP : display_8dig port map (
    clk => clk, rst => rst, bcd => mostrar(27 downto 0), neg => mostrar_neg,
    an => an, seg => seg, dp => dp);

end Behavioral;
```